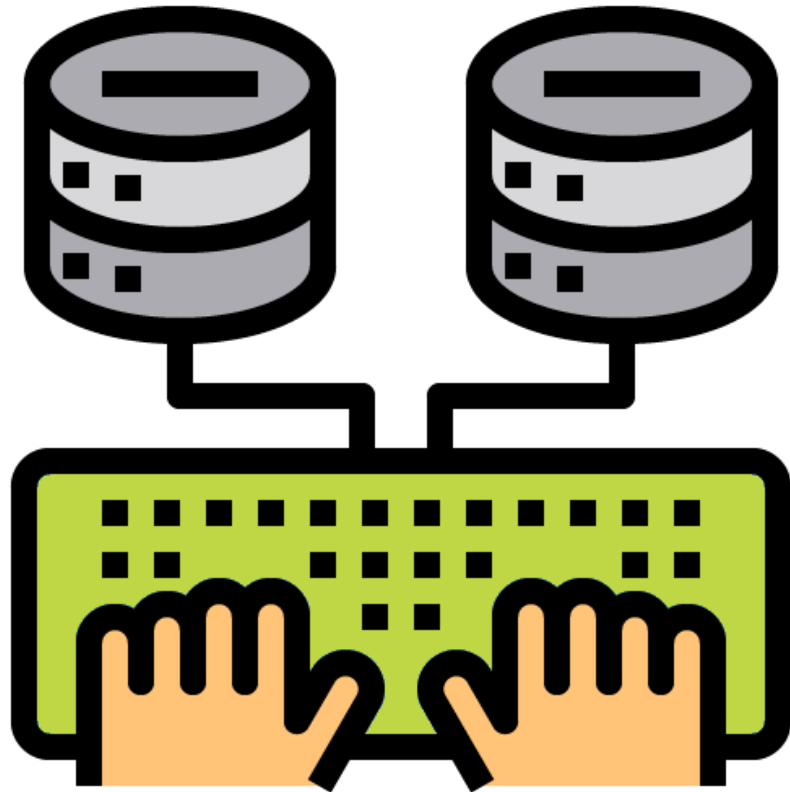




Big Data Analytics

02 - Distributed Computing

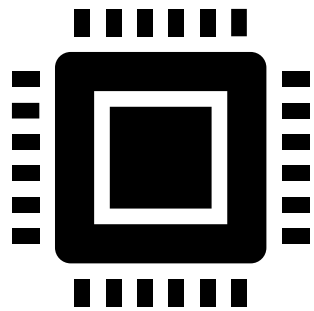
Lecturer:

Márcia Lourenço Baptista
m.baptista@novaims.unl.pt

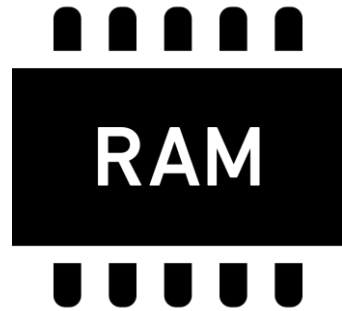
Learning Objectives

By the end of the lecture, the student should be able to:

- **Identify** Key Computing Resources
- **Explain** the Role of Processing Units (CPU & GPU)
- **Differentiate** Between Primary (RAM) and Secondary Storage (Disk)
- **Understand** Computational Scalability
- **Describe** Distributed Computing
- **Explore** intuitive Real-World Examples



Processing Unit



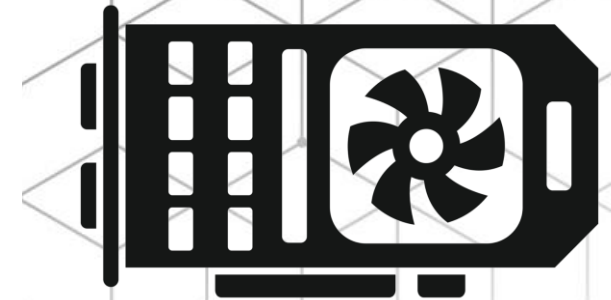
Primary Storage or
Memory



Secondary Storage
or Disk

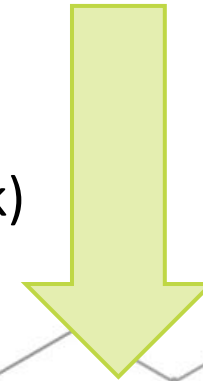


Cache Memory



GPU Processor

Abstraction



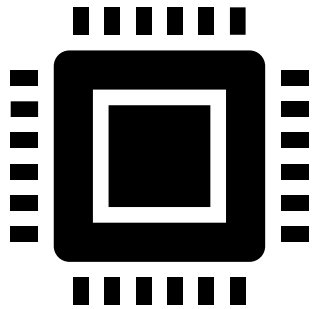
Question

- If you had to design a single computer to bring to Mars, what computing resources would you put inside, and why?

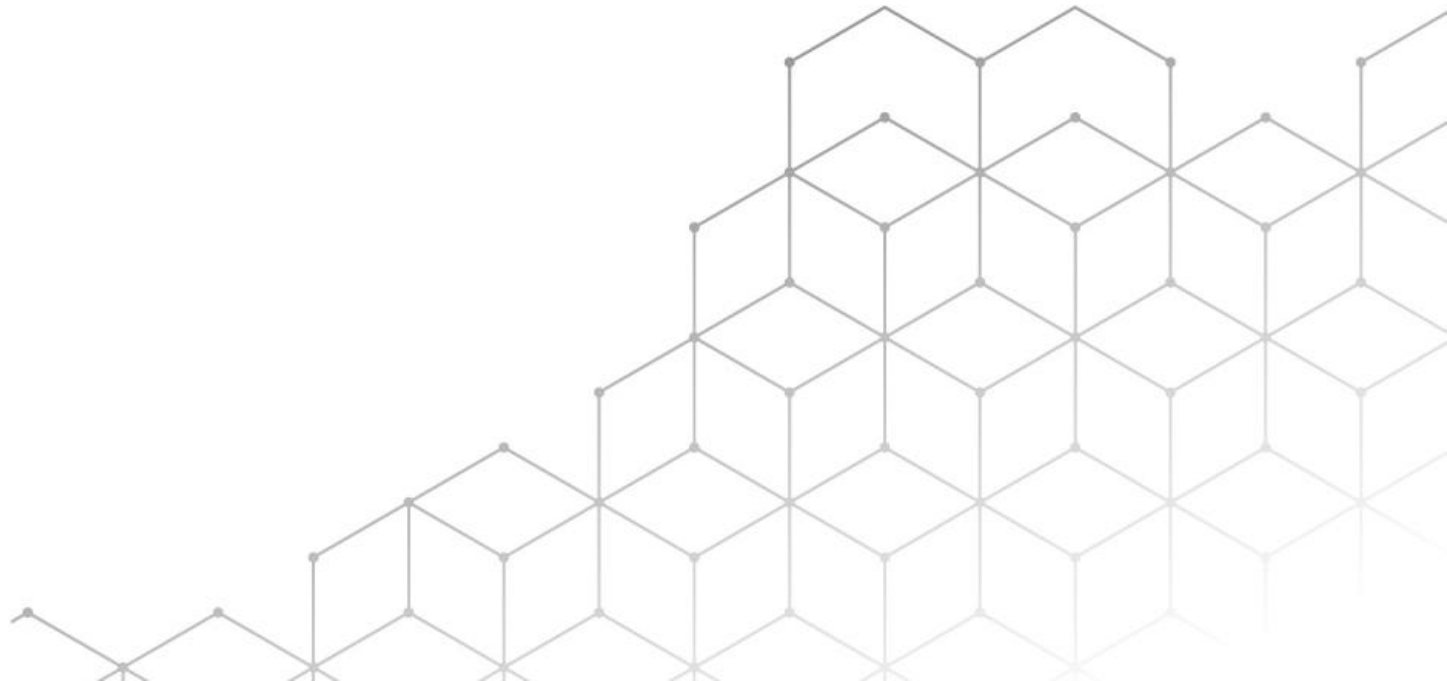


The first key piece: the Processor

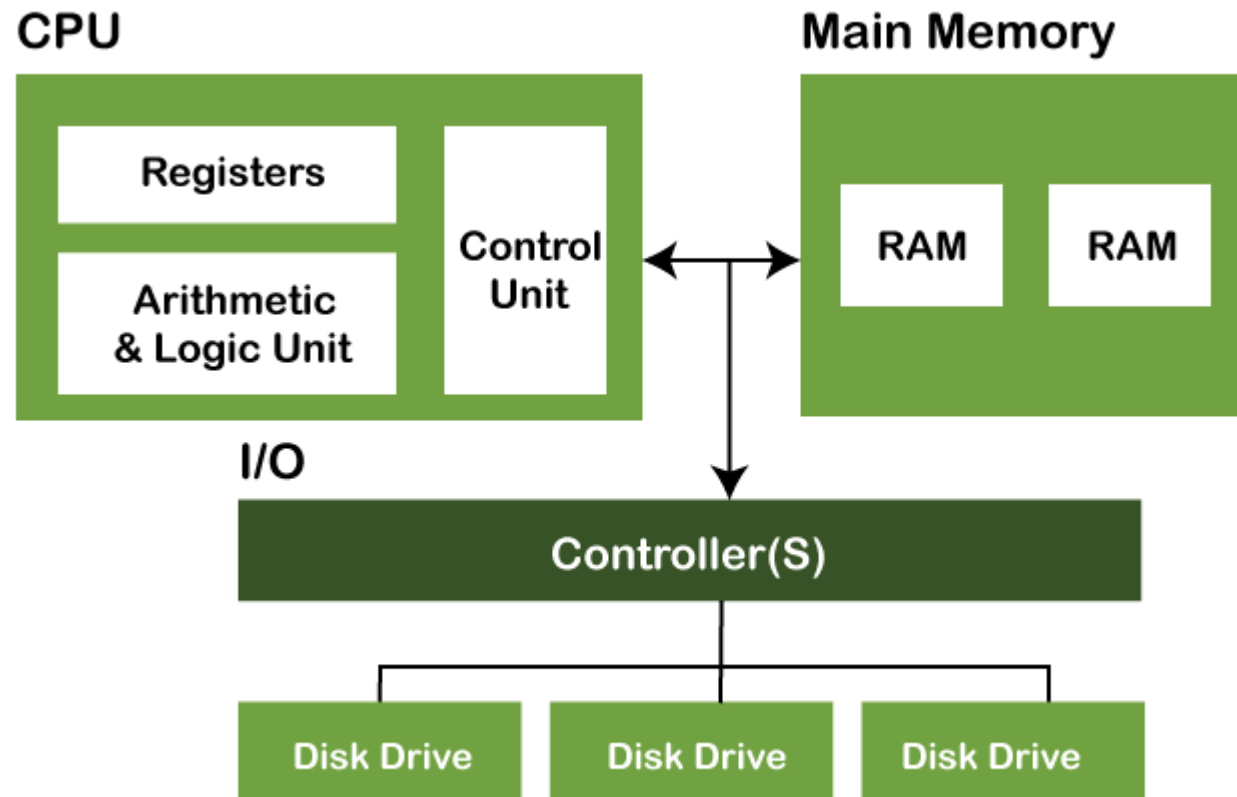
- The processor is the heart of the system
- It executes instructions, executes processes, and manages tasks to make a computer function



Processing Unit



Inside the CPU



Instructions

- These are basic commands (instructions) that CPU executes at the hardware level.

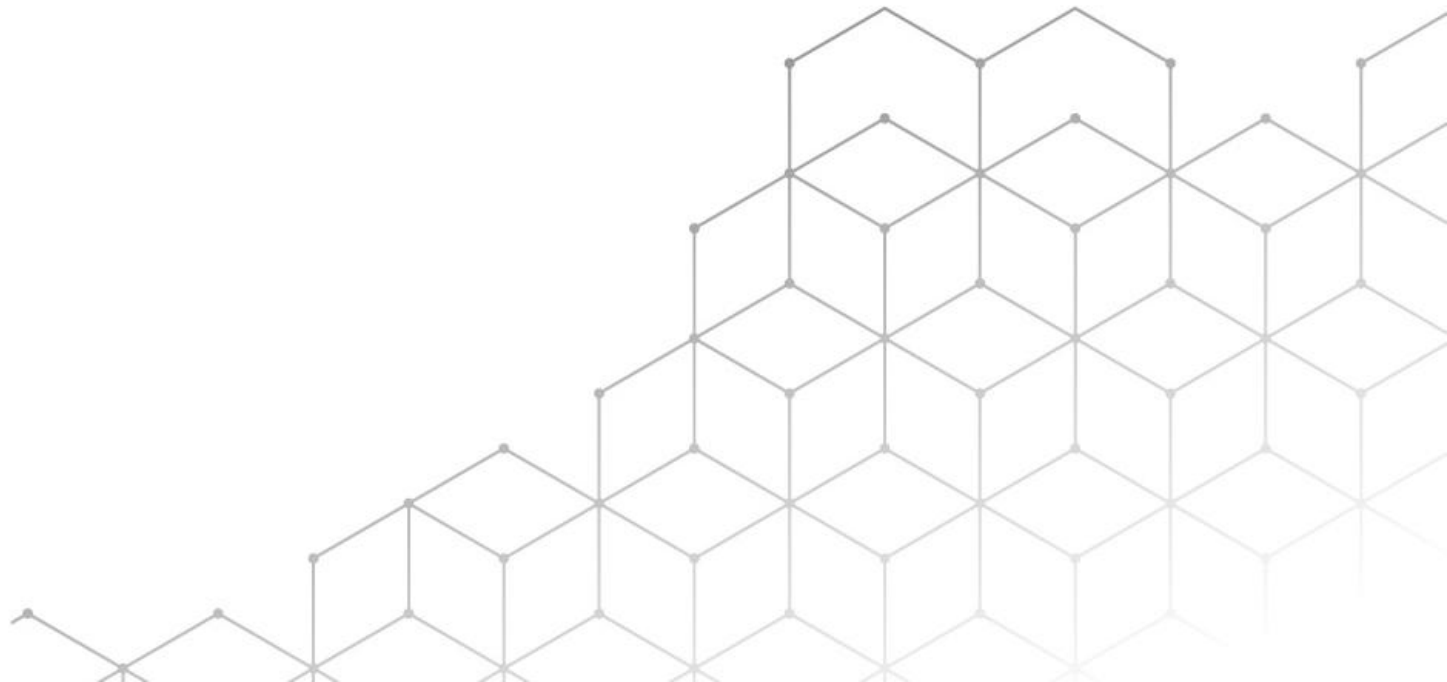
ADD R1, R2, R3

MOV R1, 5

CMP R1, R2

JMP LABEL

LOAD R1, [ADDRESS]



Instructions

- These are basic commands (instructions) that CPU executes at the hardware level.

ADD R1, R2, R3 → Adds values in registers R2 and R3 and stores in R1.

MOV R1, 5 → Moves the number 5 into register R1.

CMP R1, R2 → Compares values in R1 and R2 and outputs to the status register.

JMP LABEL → Jumps to a specific instruction (used in loops and conditionals).

LOAD R1, [ADDRESS] → Loads data from a RAM memory address into register R1.

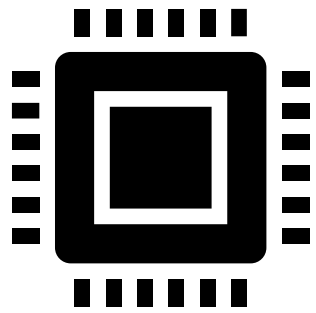
Running Processes

A **process** is a running instance of a program that consists of multiple instructions. Examples:

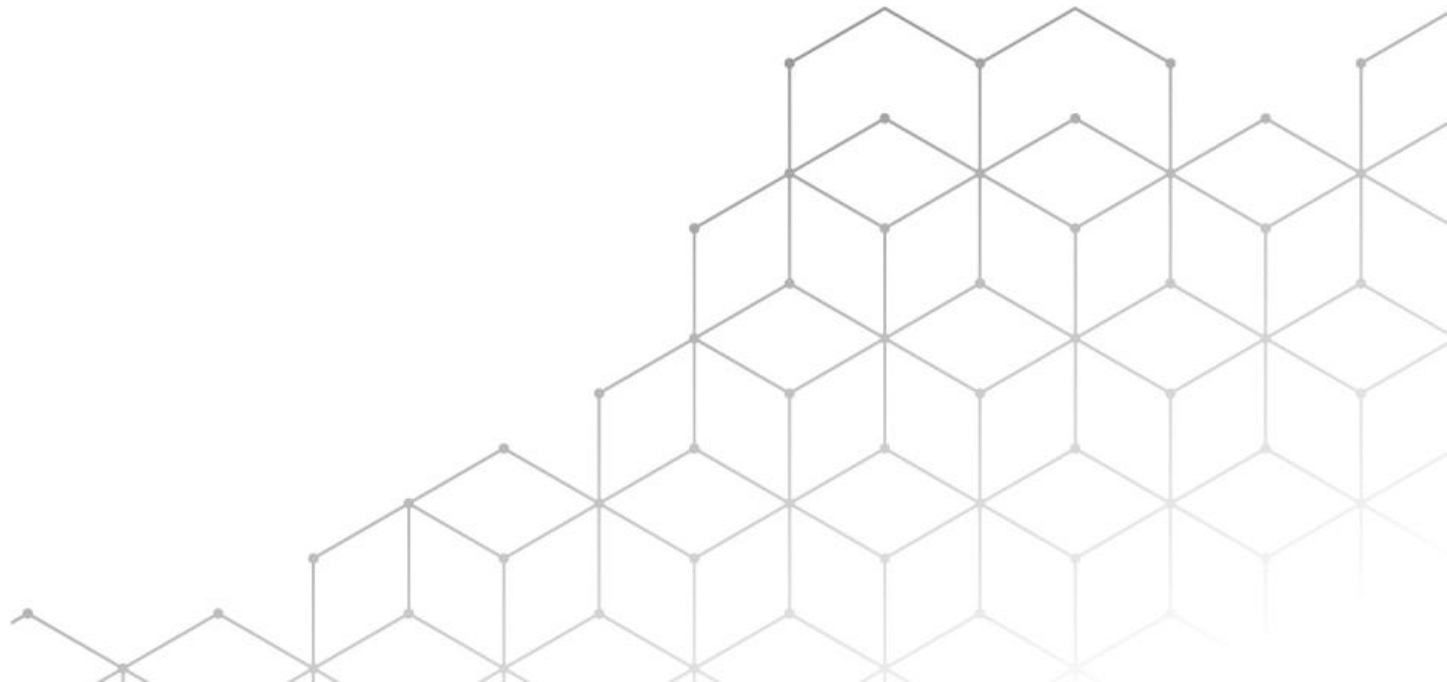
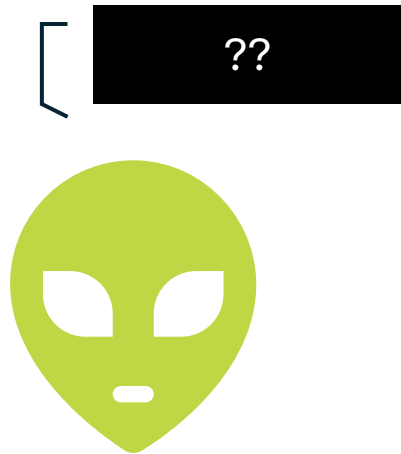
- Running a **web browser**: The CPU processes user inputs, renders pages, and fetches data from the internet.
- Running a **machine learning model**: The CPU (or GPU) loads data, performs matrix calculations, and makes predictions.
- Playing a **video game**: The CPU processes user input, game logic, and physics calculations.
- Running an **operating system (OS) process**: The CPU handles background tasks like managing memory, disk operations, and networking.

Now Back to Mars...

- What particular challenges do we have with the Processor?

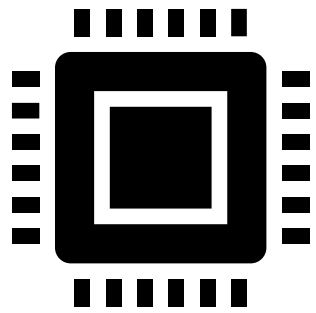


Processing Unit

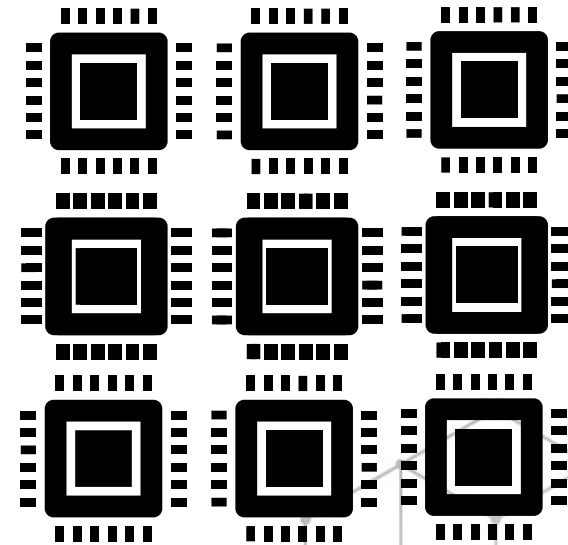
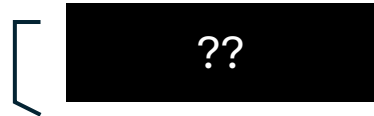


The first key piece: the Processor

- Standard processors would fail under **Martian cosmic radiation**. We need **redundancy** in case of failure.
- **Low-power** since energy on Mars is limited.

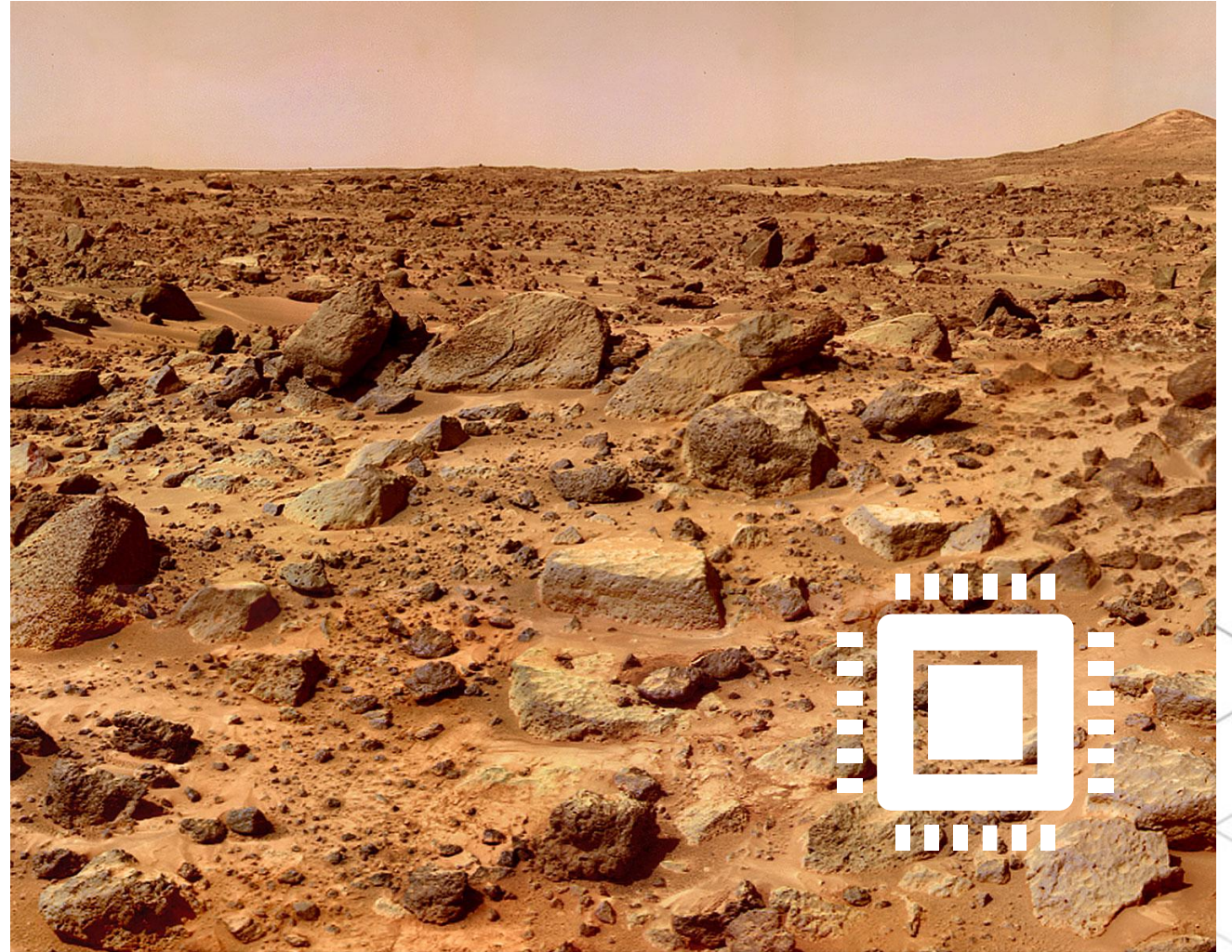


Processing Unit



What else?

Other Elements

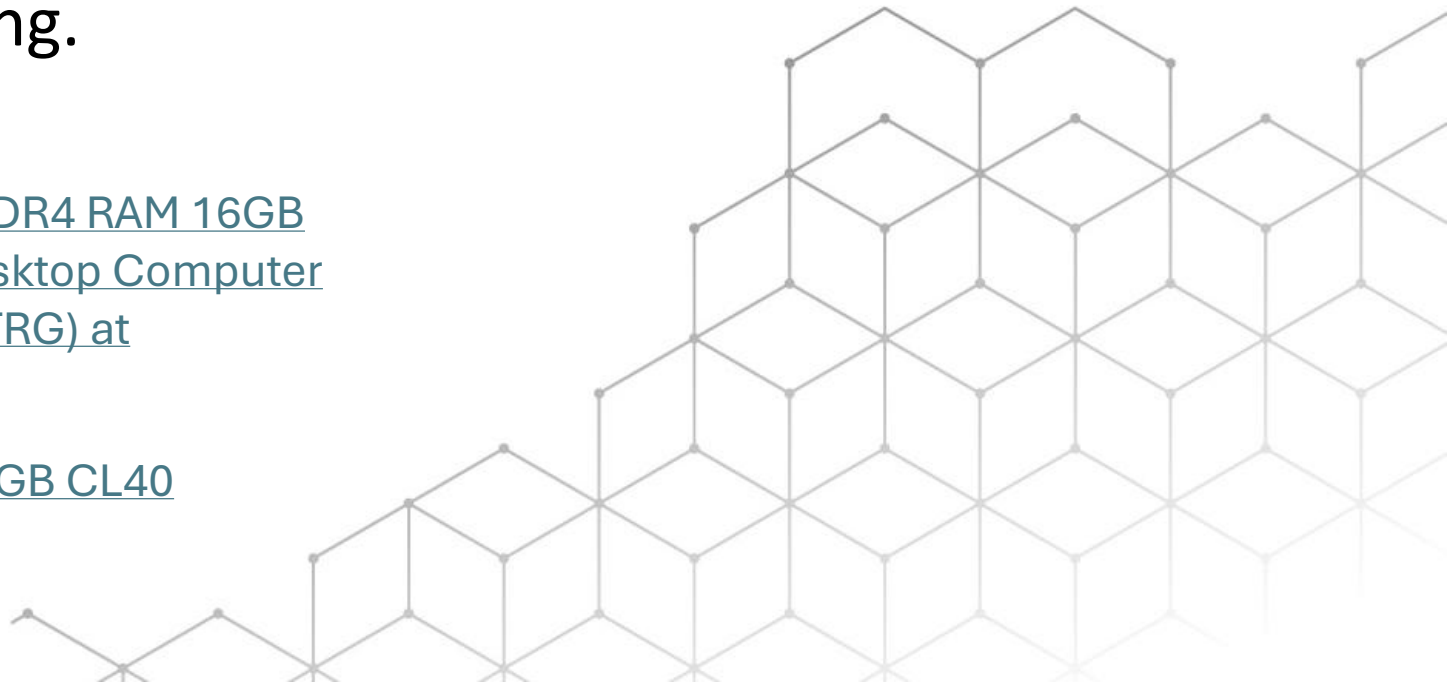


The Short Term or Primary Storage

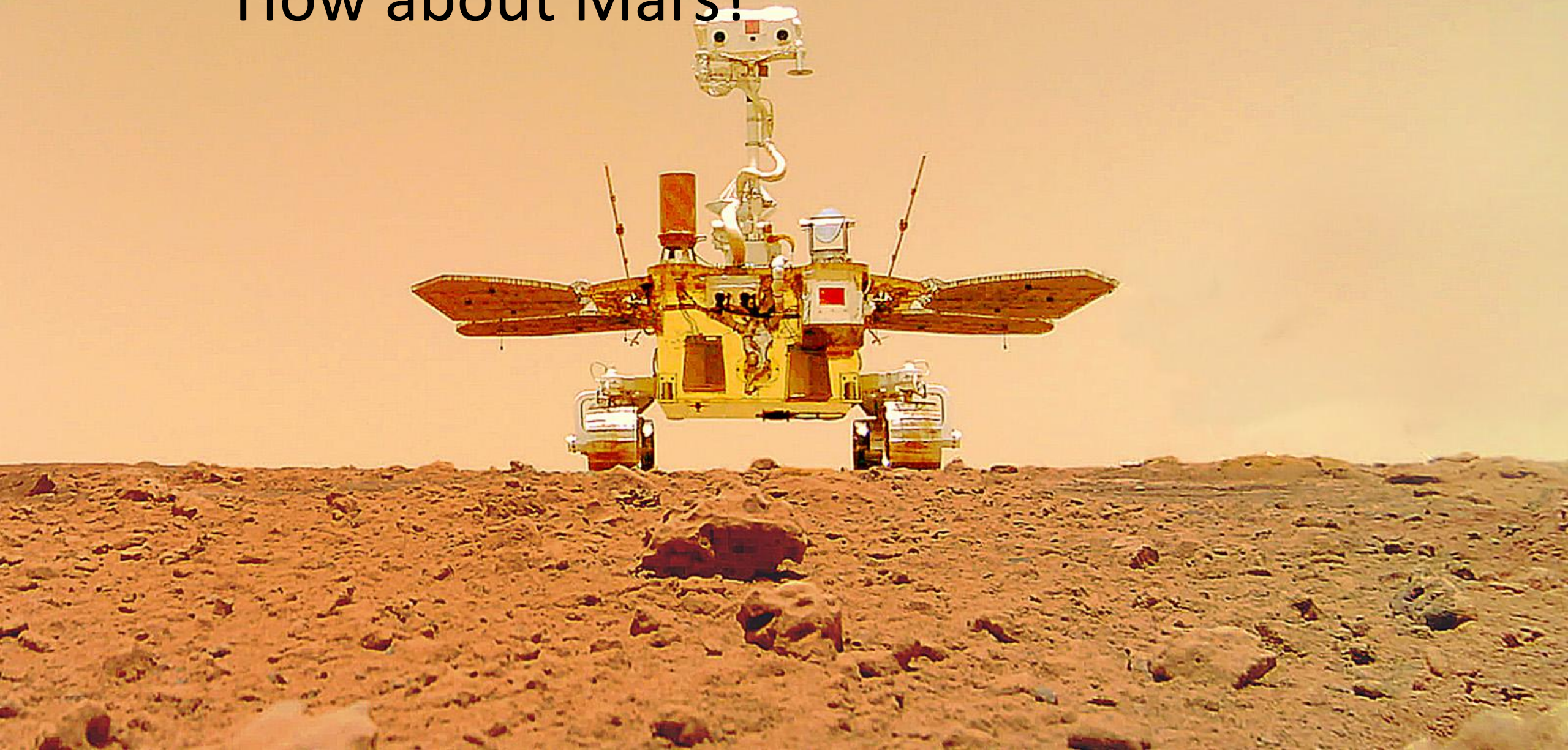
- **RAM (Random Access Memory)** is a type of computer memory that temporarily stores data that the processor is using. It allows for **fast read and write**.
- **RAM is Primary Storage** because it works directly with the CPU and is used for immediate processing.

[G.SKILL Trident Z Royal Series \(Intel XMP\) DDR4 RAM 16GB \(2x8GB\) 3200MT/s CL16-18-18-38 1.35V Desktop Computer Memory UDIMM - Gold \(F4-3200C16D-16GTRG\) at Amazon.com](#)

[Kingston Server Premier DDR5 4800MHz 64GB CL40 Memória para Servidor Hynix M Rambus | PcComponentes.pt](#)

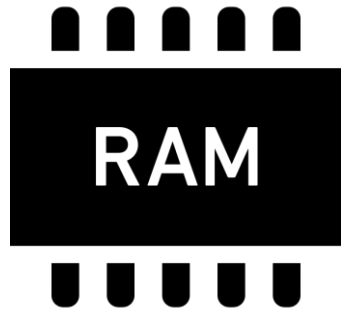


How about Mars?



Back to Mars...

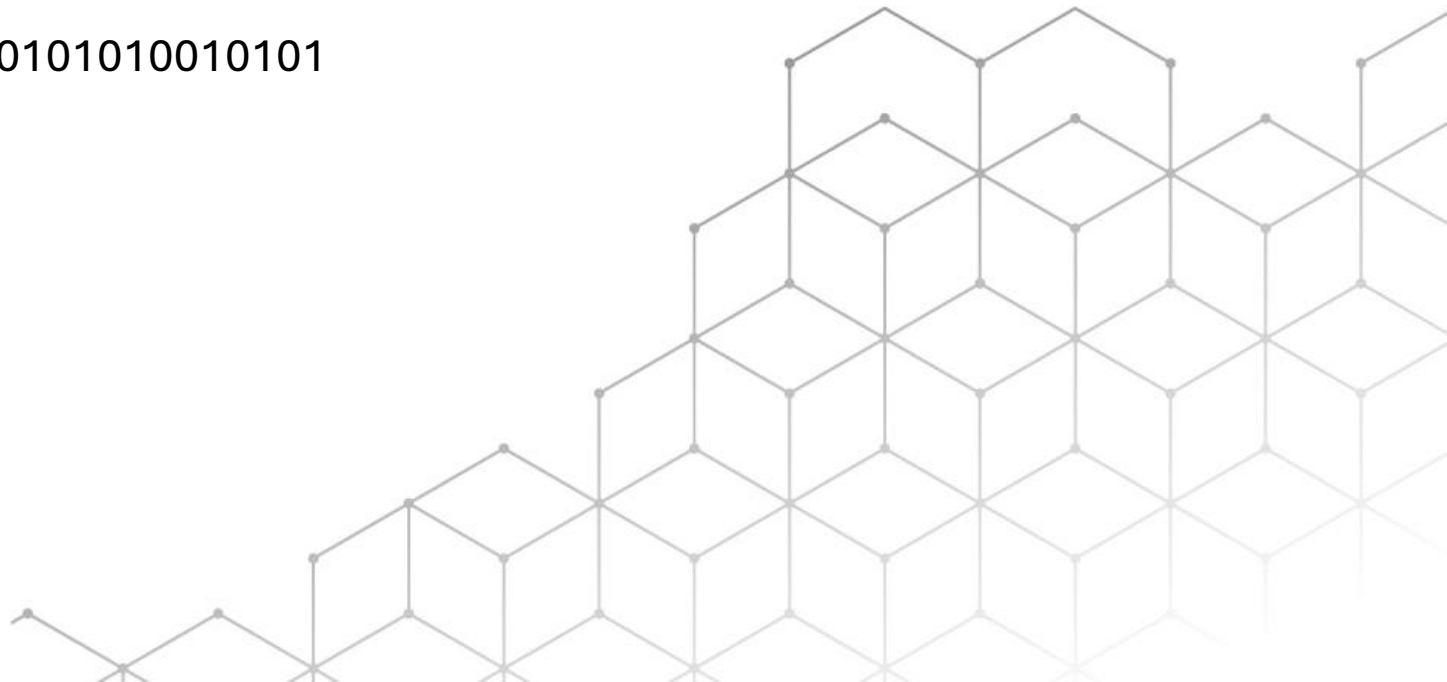
- **Memory Protection:** need to handle bit flips from radiation.



Primary Storage or
Memory



0010101010010101



The “Forever” or Secondary Storage

- Disk storage refers to **permanent storage** to retain data even when the power is turned off (non-volatile).



Primary Storage or
Memory



How about Mars?



Back to Mars...

- Radiation-hardened (bit flips), redundant copies
- Low-power since energy on Mars is limited.

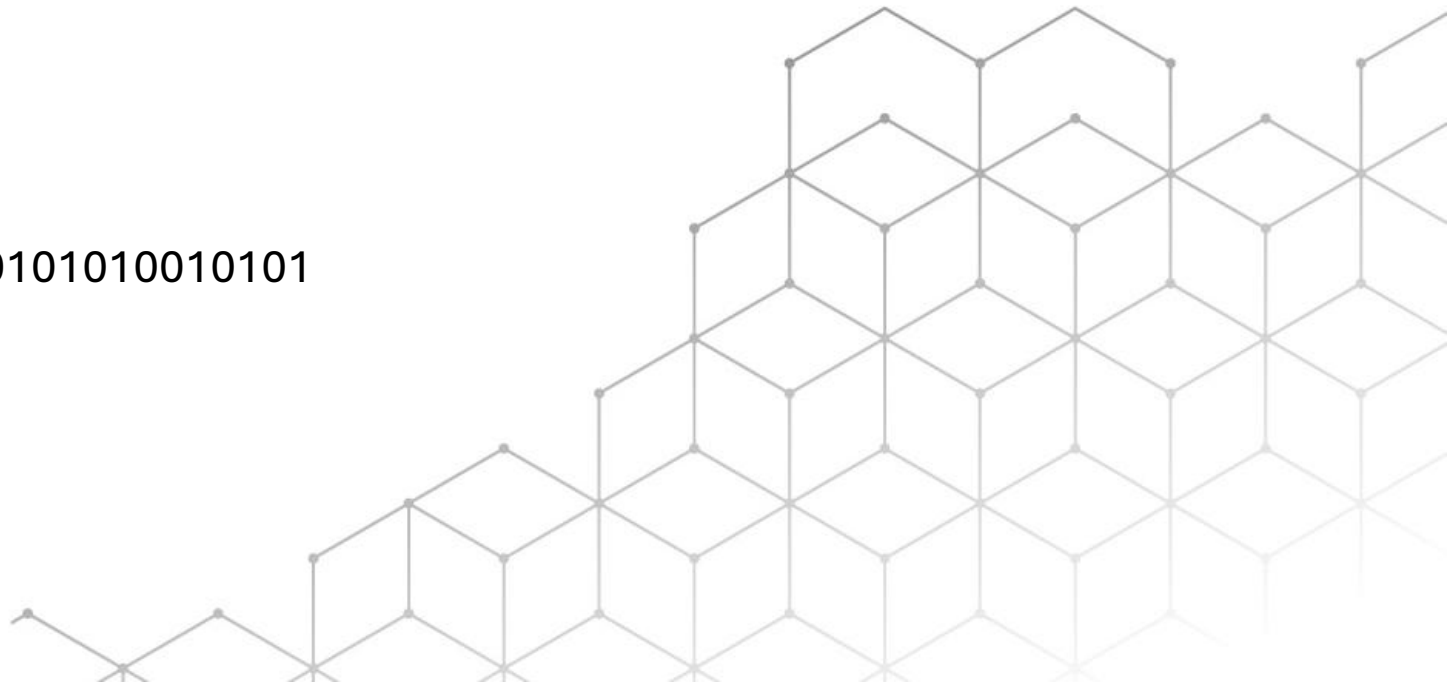


**Primary Storage or
Memory**



I am slow.

0010101010010101



NOVA

IMS

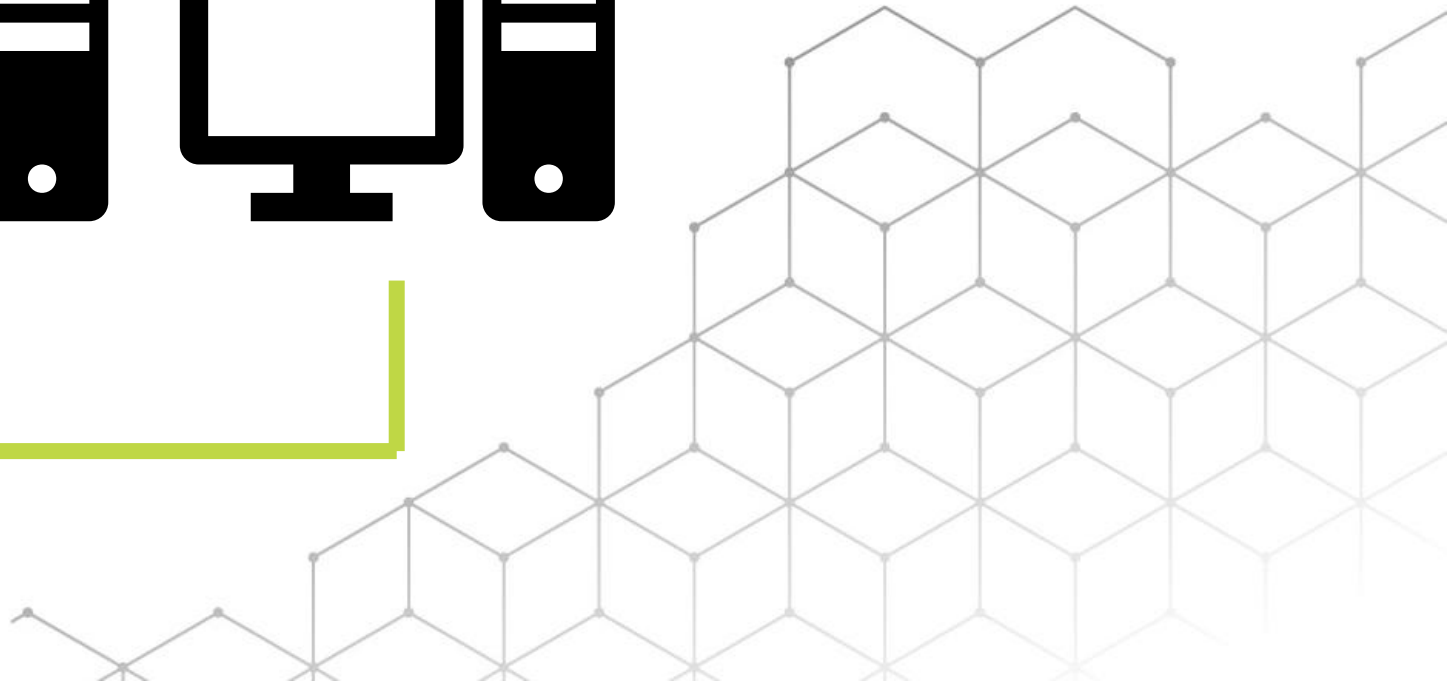
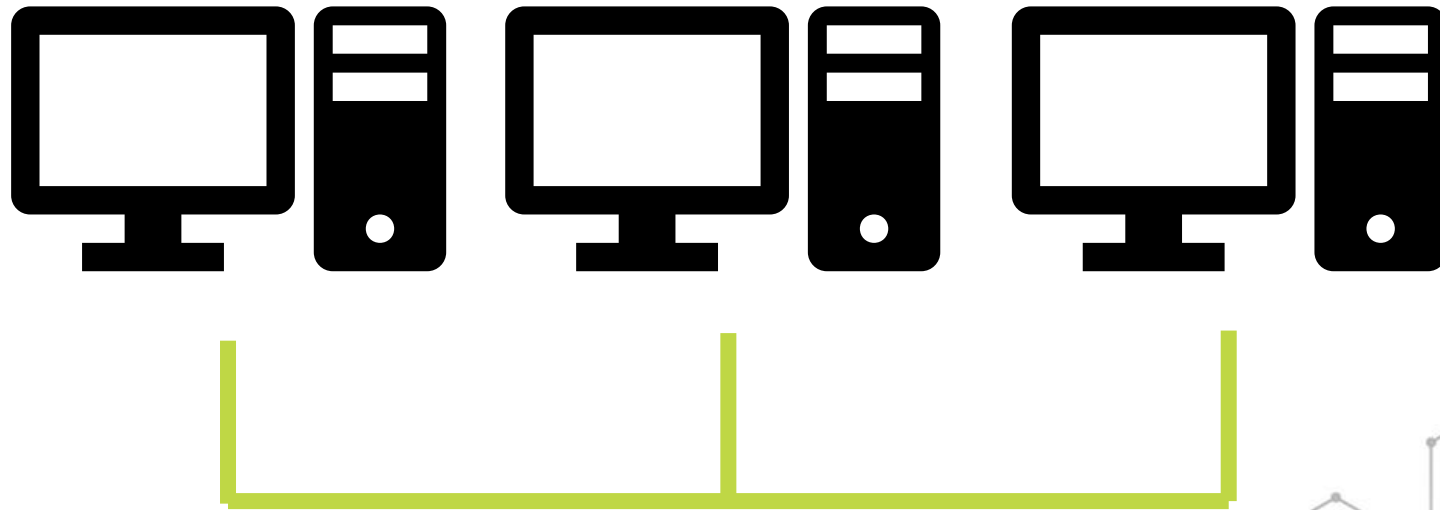
Information
Management
School

What about a Distributed System?



What is a Distributed System

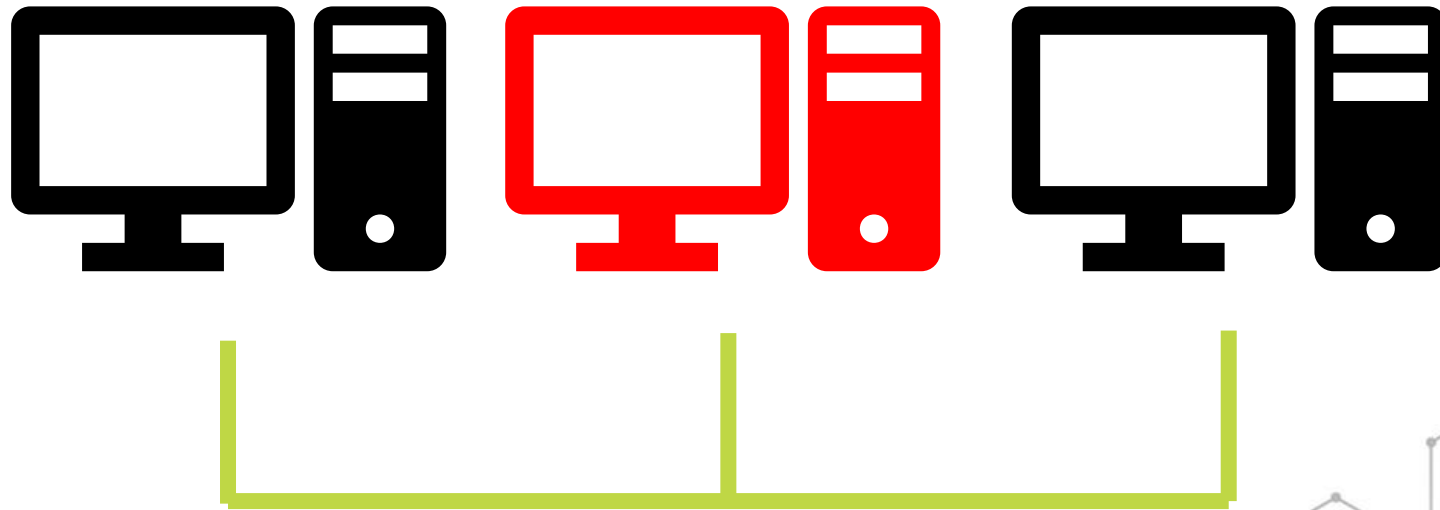
- A collection of interconnected computers that work together to achieve a common goal



Fault Tolerance

Fault Tolerance & Redundancy

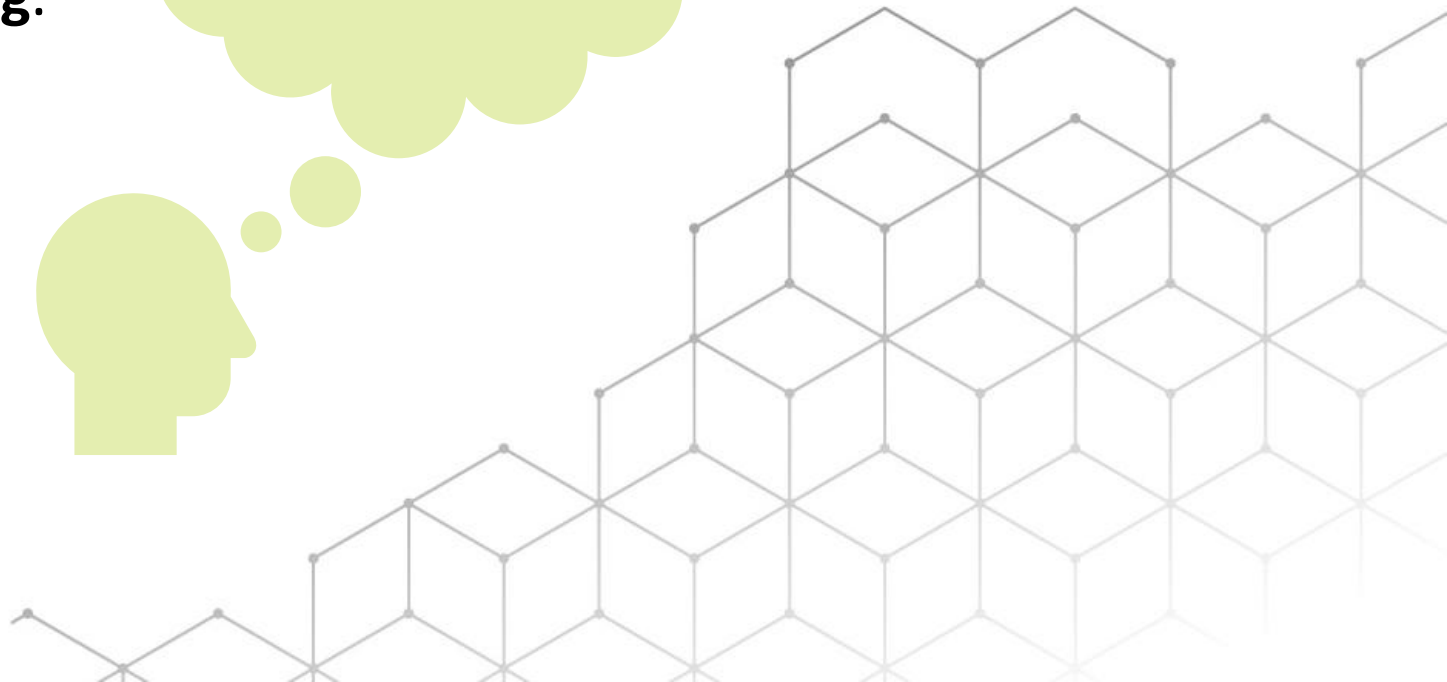
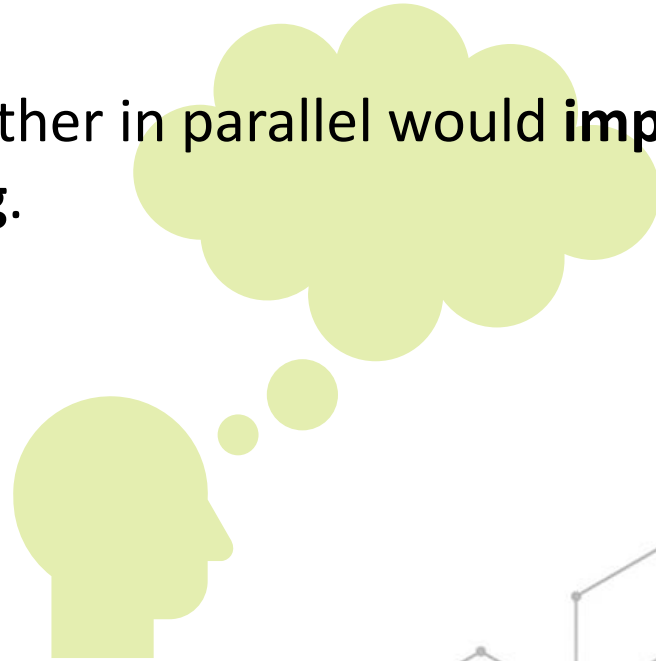
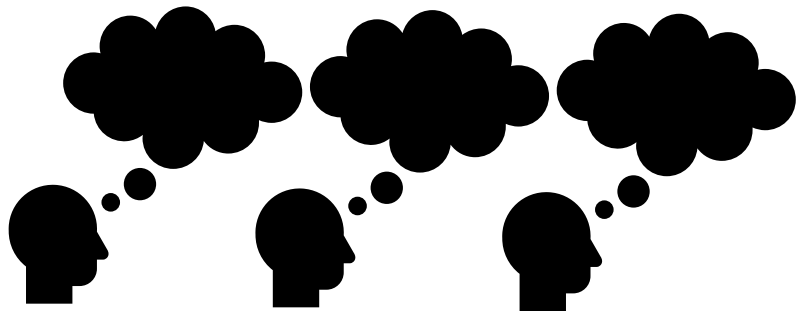
- A single computer is a **single point of failure**.
- A distributed system ensures **backup nodes can take over** if one fails.



Parallel Processing

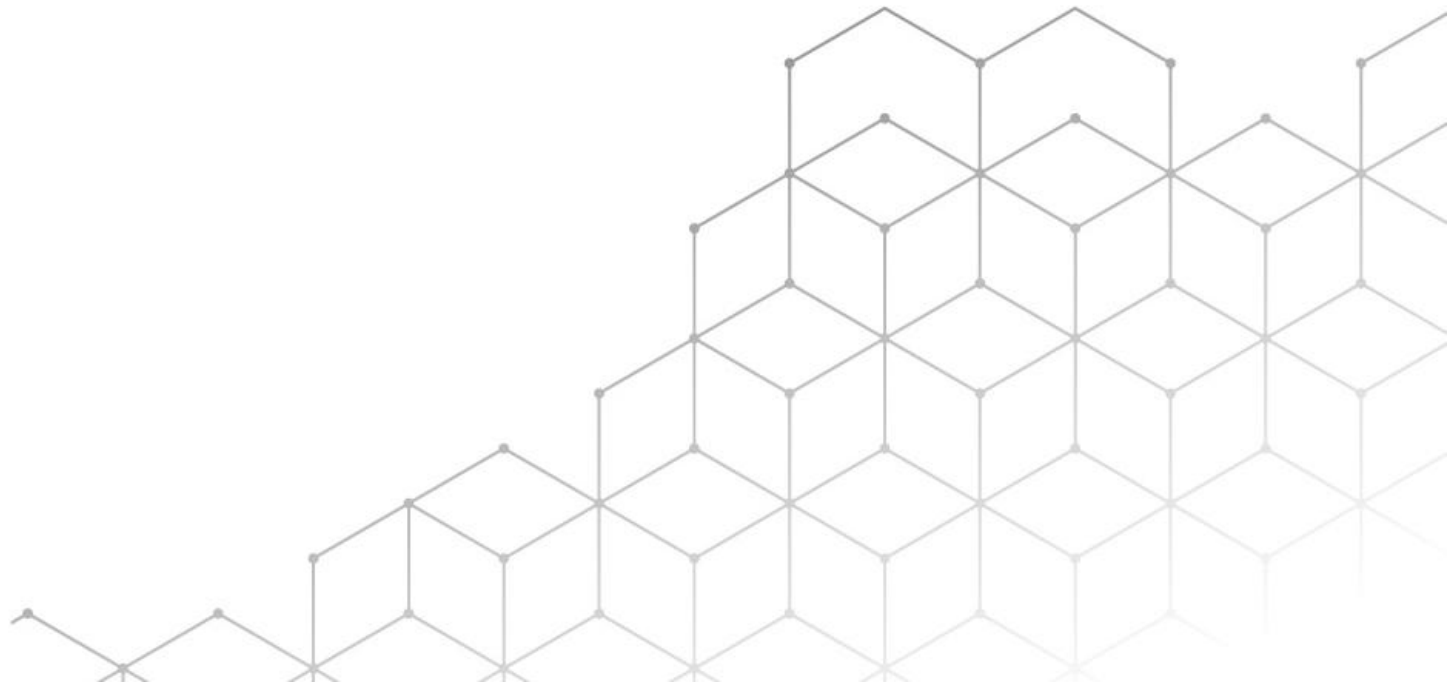
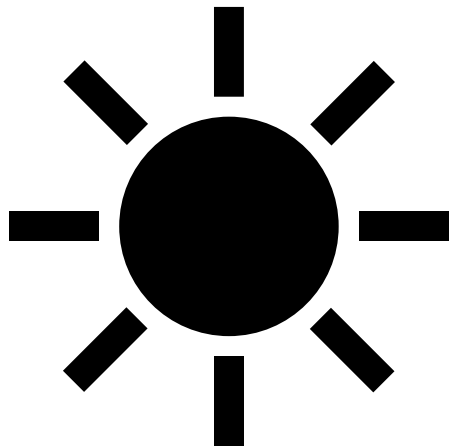
Parallel Processing & Efficiency

- Tasks like **AI-based navigation, terrain mapping, and scientific analysis** require heavy computation.
- Multiple systems working together in parallel would **improve performance and speed up decision-making.**



Energy Management

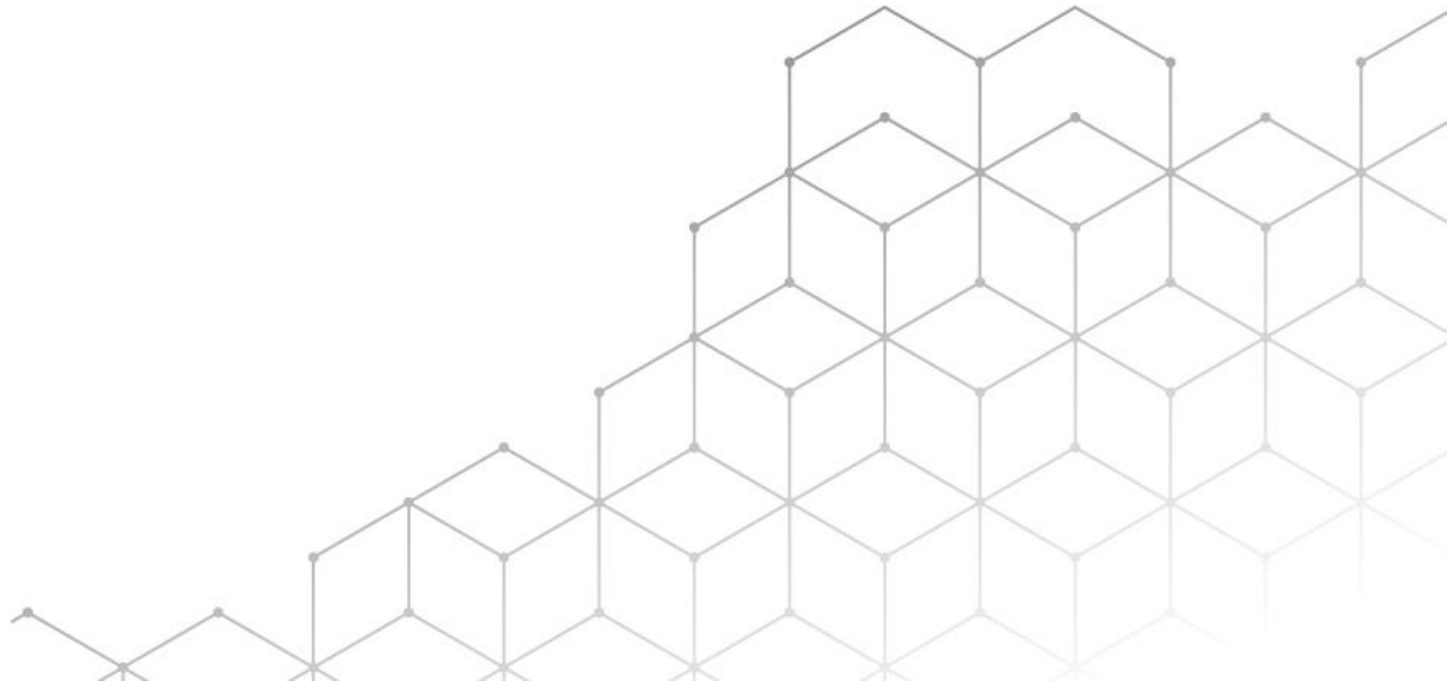
- A single powerful computer requires **high energy** and cooling, which is challenging on Mars.
- Distributed nodes can **turn on/off based on energy availability** (e.g., using solar power intelligently).



Scalability

If a **Mars colony or research station** is established, new computers/sensors can integrate seamlessly into a distributed network.

Scalability is the ability to **grow larger** as a system



Space Dream



What about the Challenges?

- Maintaining concurrency
- Overcoming the lack of a global clock
- Managing the independent failure of components
- Heterogeneity
- Scalability
- Transparency
- Security



What about the Challenges?

- Maintaining concurrency
- Overcoming the lack of a global clock
- Managing the independent failure of components
- Heterogeneity
- Scalability
- Transparency
- Security



Thank you
DSAA

NOVA

IMS

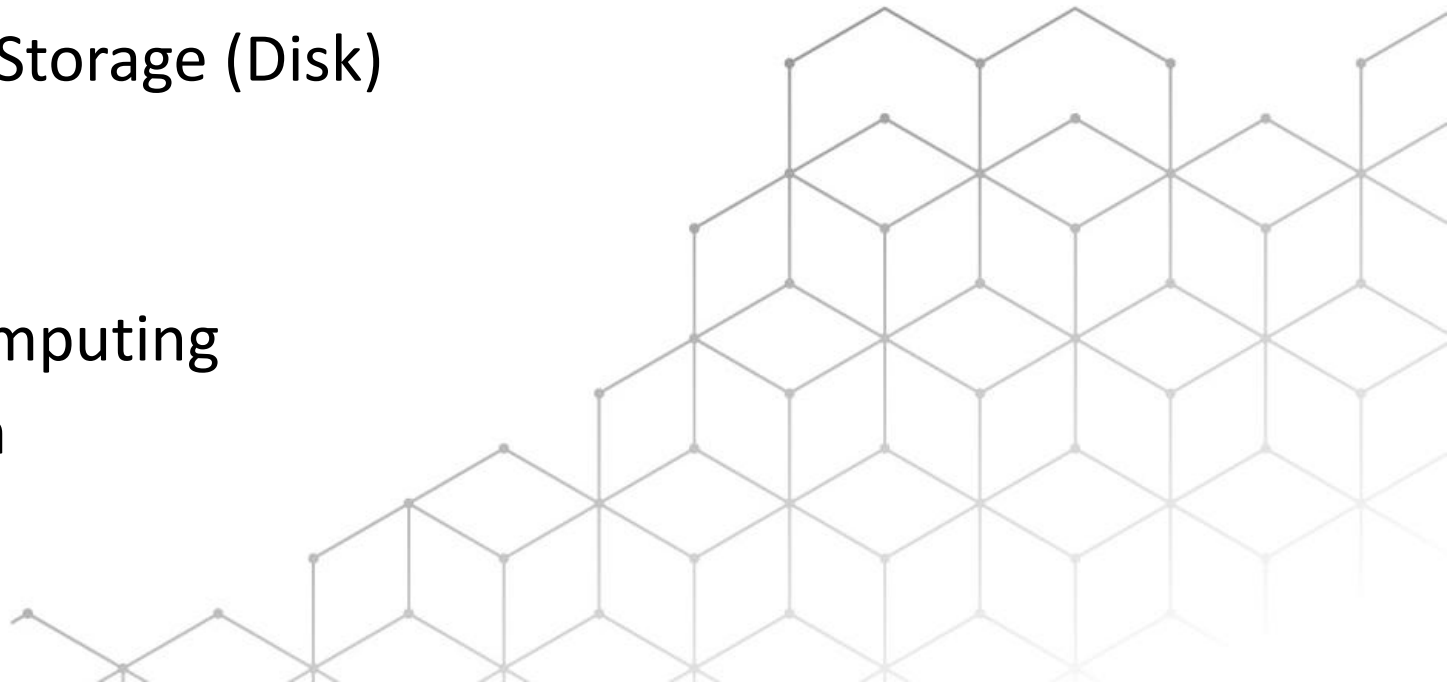
Information
Management
School



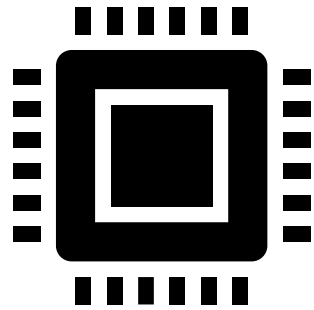
data with **purpose**

Agenda for Today...

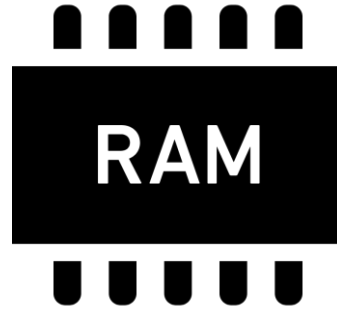
- Computing Resources
 - Essential computing resources
 - Processing power
 - Storage (RAM memory, cache, and disk storage)
- Role of the Processing Unit (CPU & GPU)
- Primary (RAM) and Secondary Storage (Disk)
- Computational Scalability
- Distributed Computing
- Components of Distributed Computing
- Computing Resources in Action



Computing Resources



Processing Unit



Primary Storage

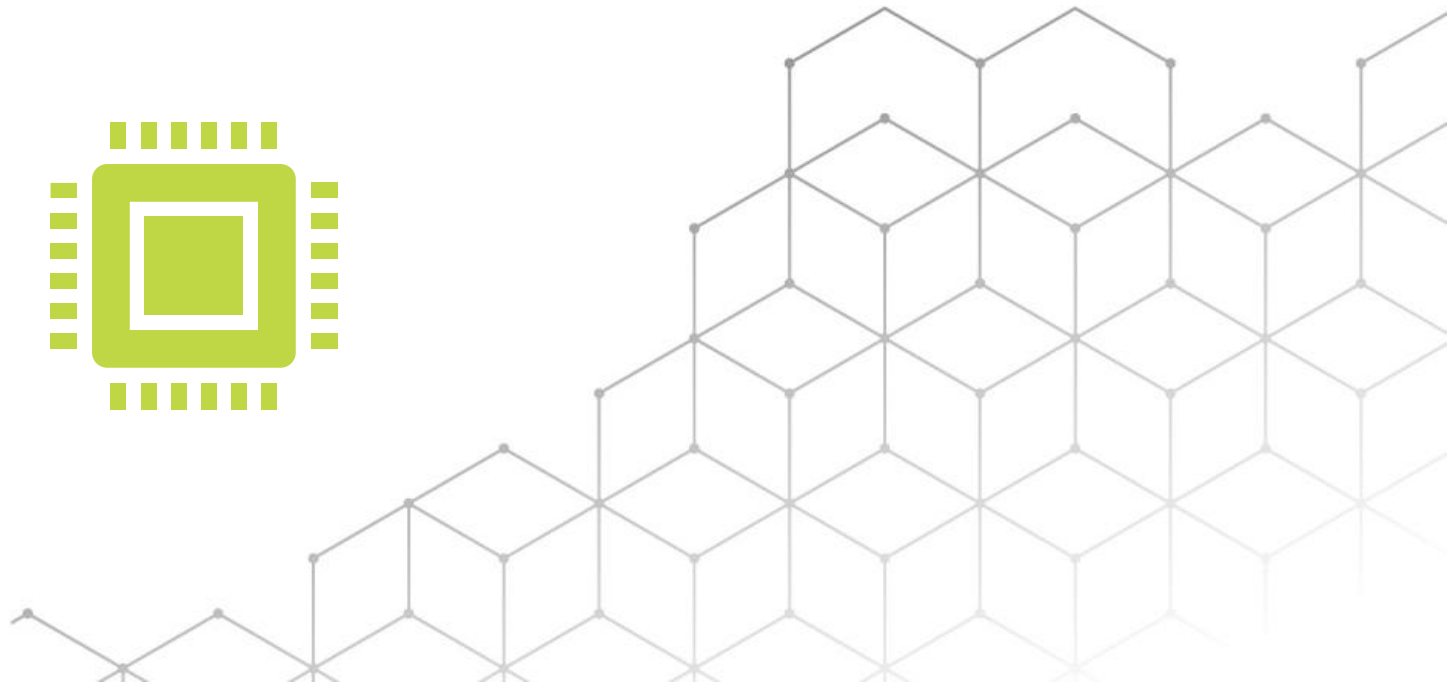
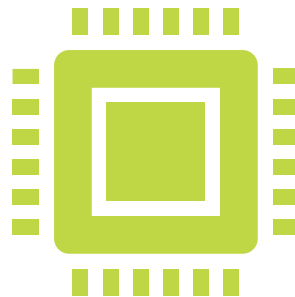
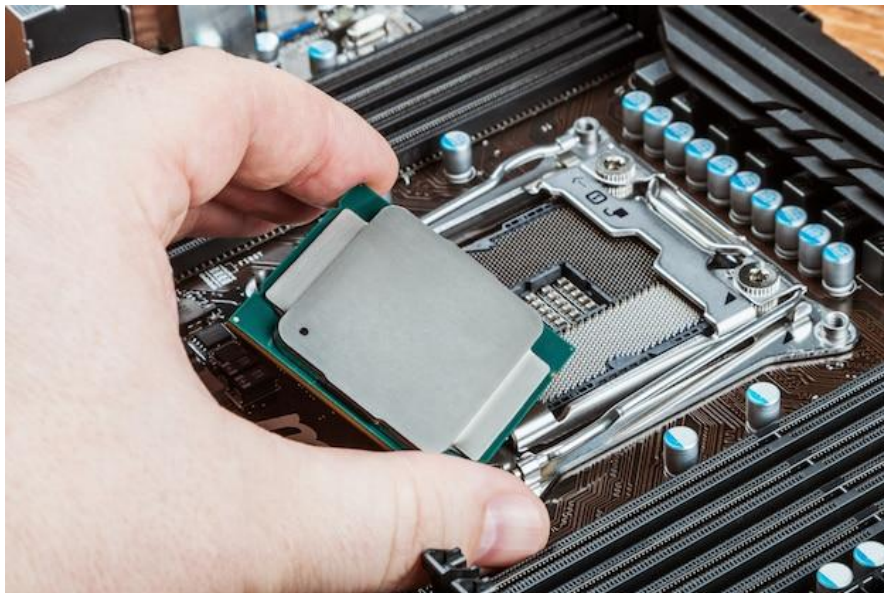


Secondary Storage



Processing Unit

- A **processor** is the primary computing resource that executes instructions and processes data.
- It is the **brain of a computer**.



Processor Types



CPU



GPU



TPU

1. CPU (Central Processing Unit)

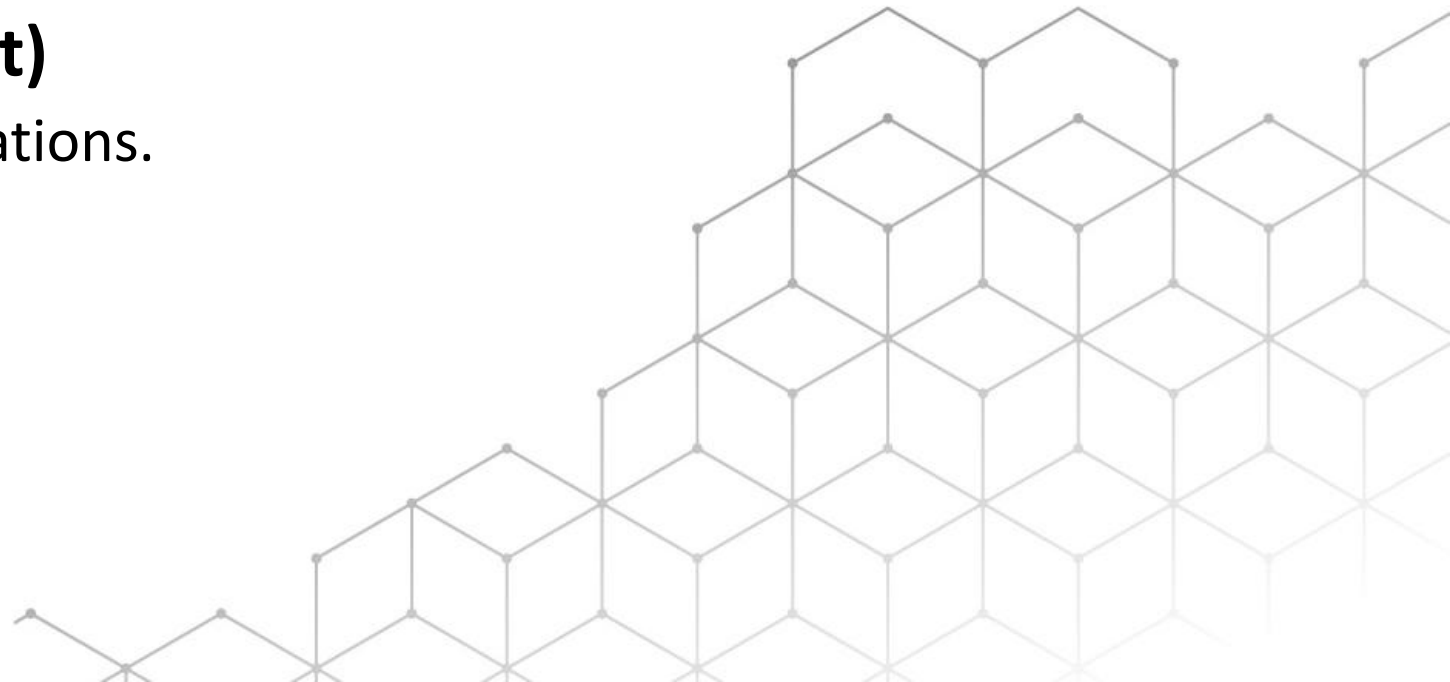
- General-purpose processing.

2. GPU (Graphics Processing Unit)

- Efficient for graphics rendering and AI workloads.

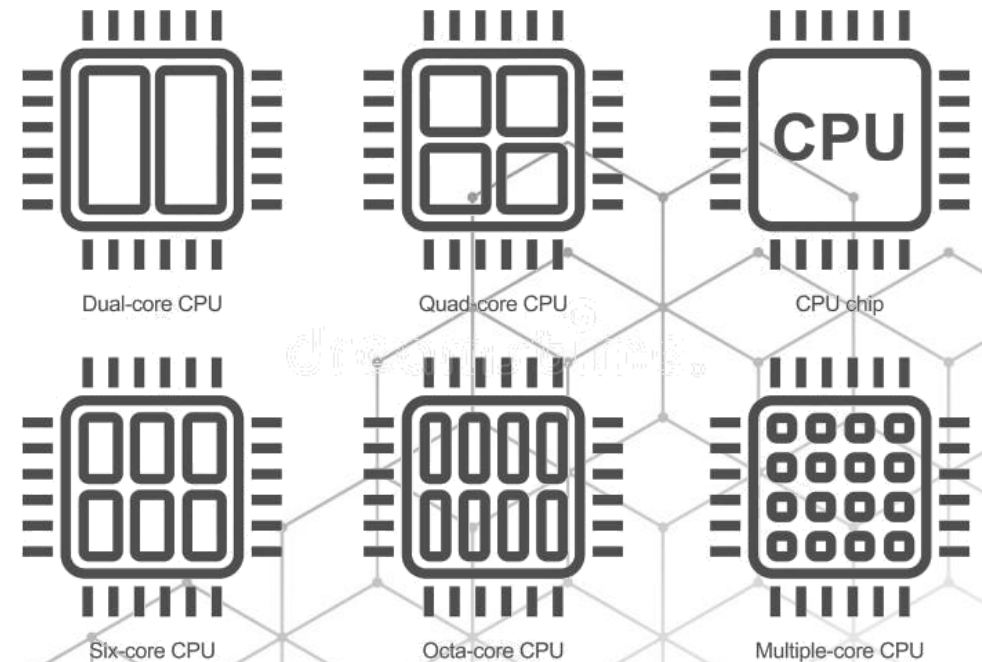
3. TPU (Tensor Processing Unit)

- Optimized for tensor computations.



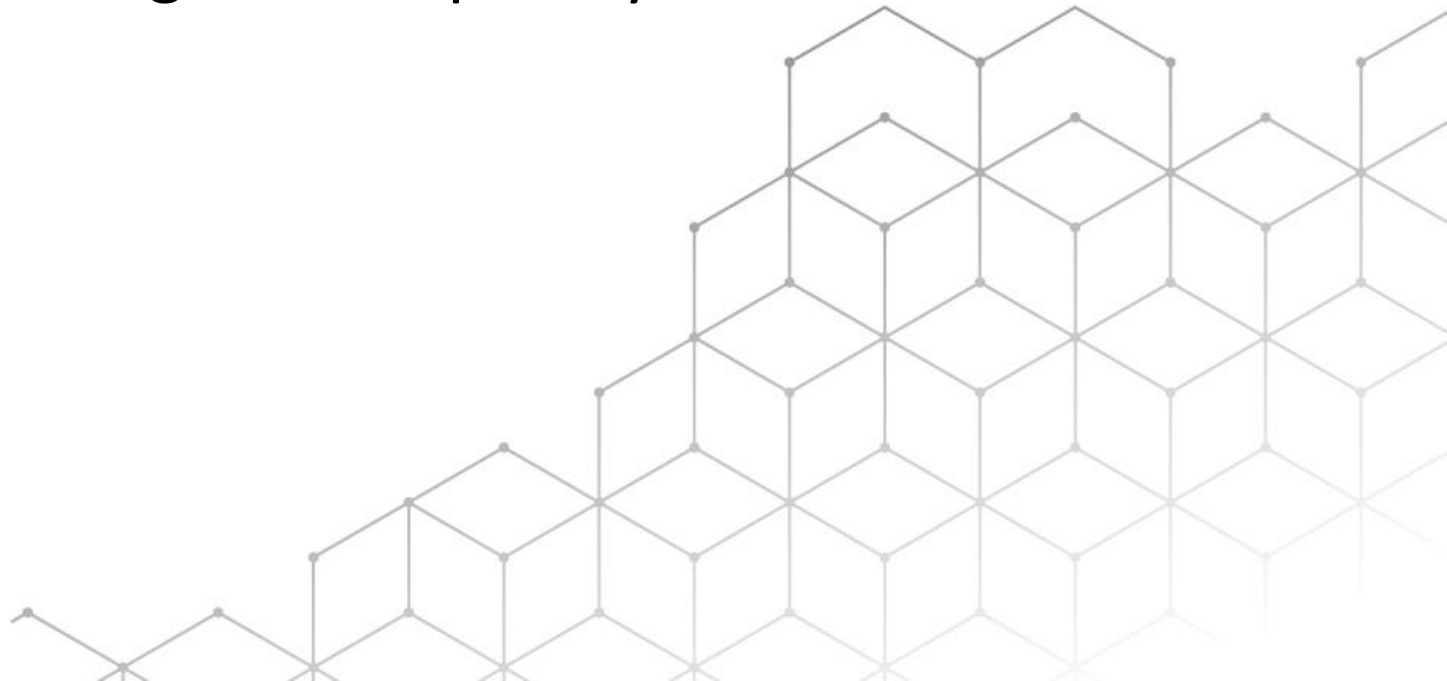
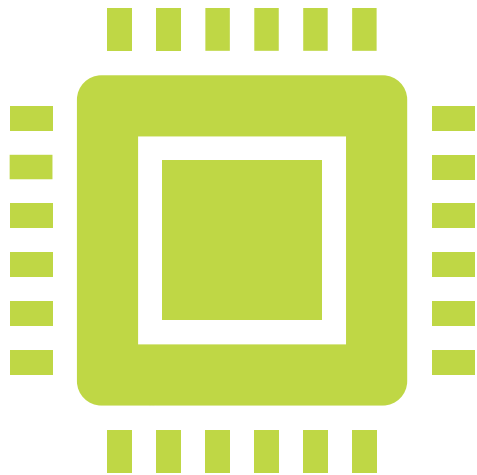
Multi-core Processor

- **Multiple processing units (cores) on a single chip.**
- Each core acts as an independent processor that can execute tasks simultaneously.
- Good for parallel processing
- GPU is **multiple-core** or **single-core**?



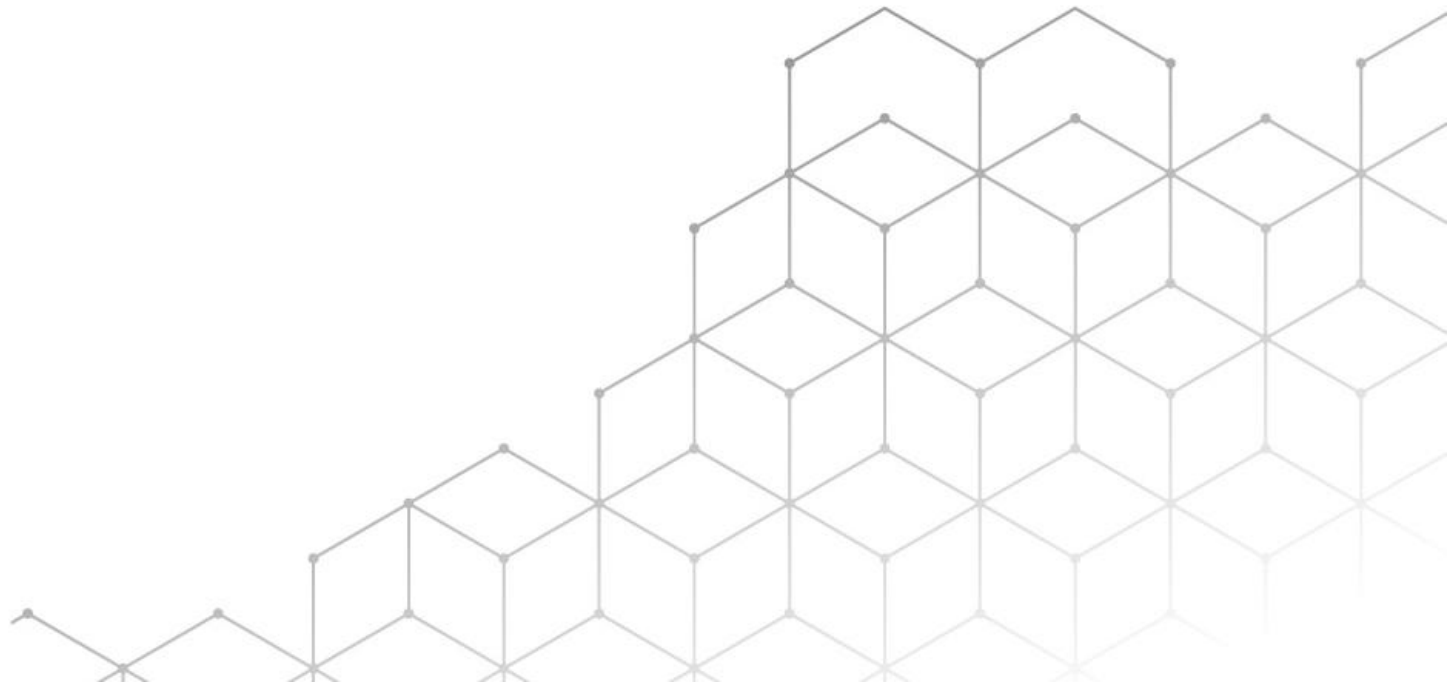
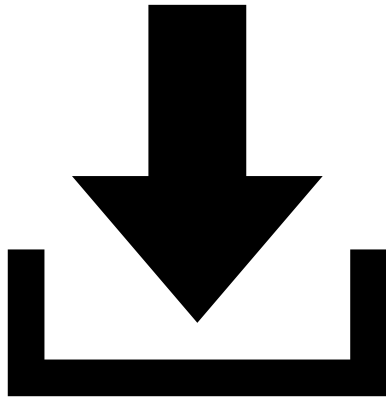
Key Features of a Processing Unit

- **Processing Speed:** Measured in GHz, determines execution speed.
- **Multi-Core Architecture:** Enables parallel processing (Dual-core, Quad-core, etc.).
- **Cache Memory:** High-speed storage for frequently used data.



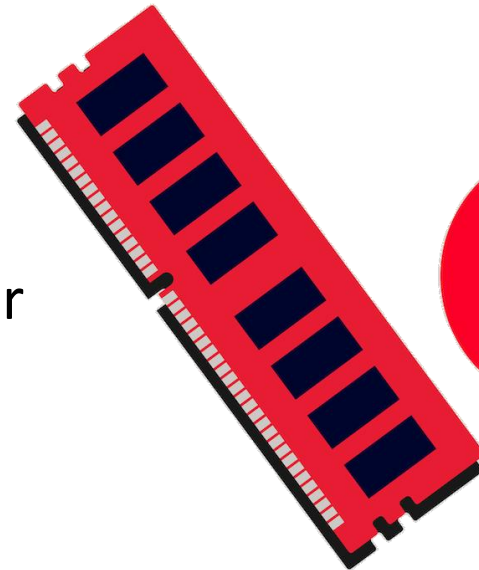
Storage

- **Storage** is a computing resource responsible for **saving, retrieving, and managing data**. It enables both temporary and permanent data retention.



Types of Storage

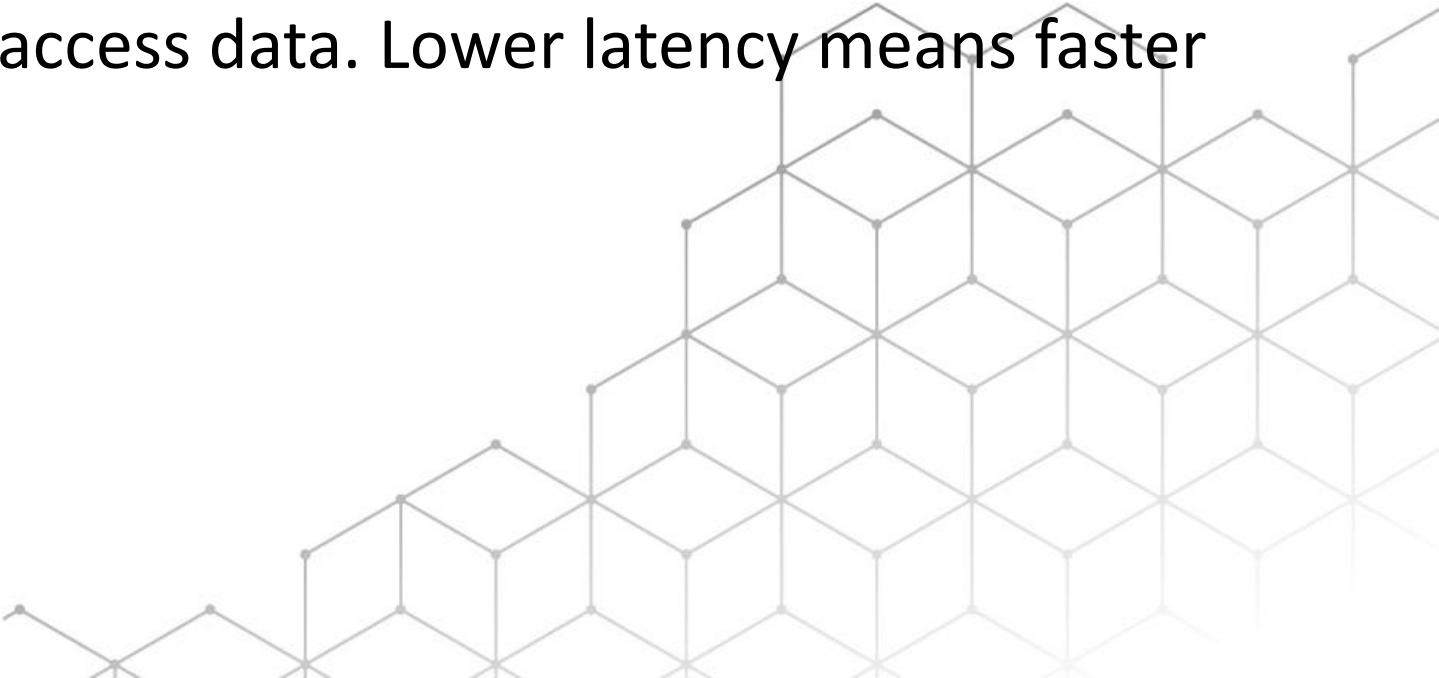
- **Primary Storage (Volatile):**
 - RAM (Random Access Memory) provides temporary, fast access for active processes.
- **Secondary Storage (Non-Volatile):**
 - HDDs (Hard Disk Drives) and SSDs (Solid-State Drives) store data persistently.
- **Cloud Storage:**
 - Remote, scalable, accessible storage over the internet.



WHAT'S
the
DIFF?

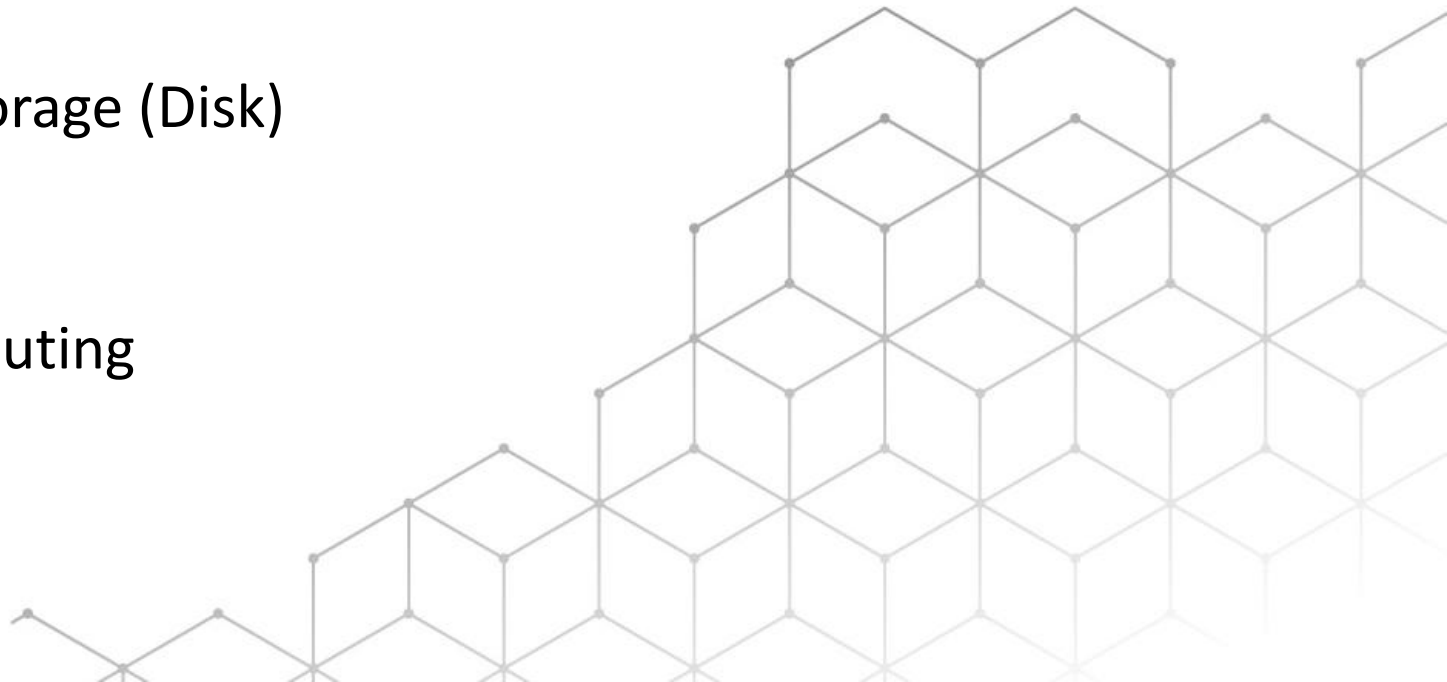


Storage Performance Metrics

- **Speed:** Measured in read/write speeds (MB/s, GB/s). SSDs are much faster than HDDs.
 - **Capacity:** Determines how much data can be stored, ranging from gigabytes to petabytes.
 - **Latency:** The time it takes to access data. Lower latency means faster performance.
- 

Agenda for Today...

- Computing Resources
- Role of the Processing Unit (CPU & GPU)
 - Questions
 - CPU vs Human Brain
 - CPU vs GPU
 - Graphical Processing Units
 - Deep Learning vs GPUs
- Primary (RAM) and Secondary Storage (Disk)
- Computational Scalability
- Distributed Computing
- Components of Distributed Computing
- Computing Resources in Action



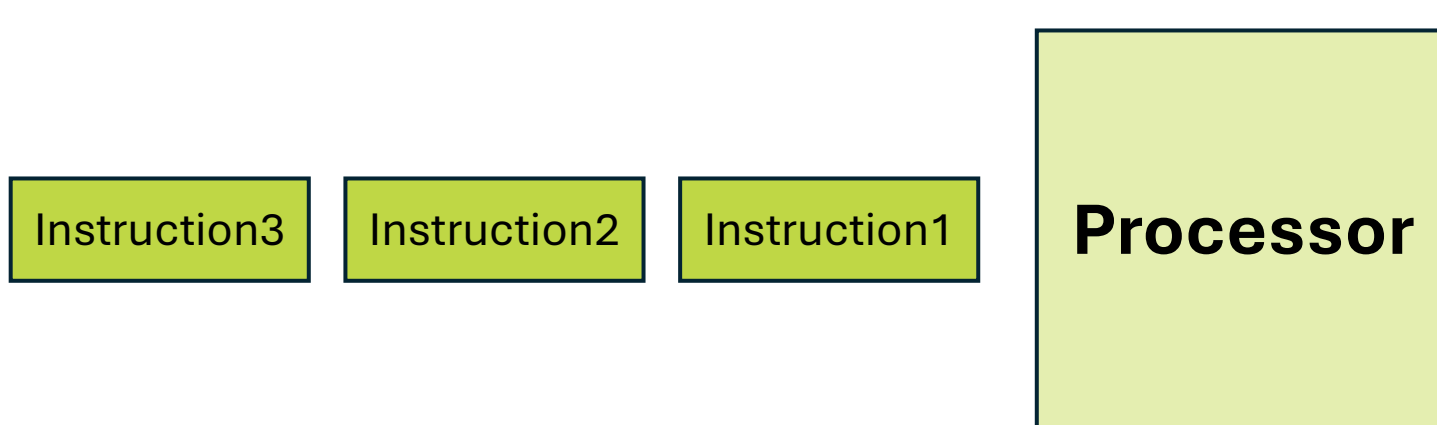
Questions

- How does a CPU process information differently from a human brain?
- If CPUs are so powerful, why do we even need GPUs?
- Why do gamers and AI researchers care so much about GPUs?



Sequential Processing

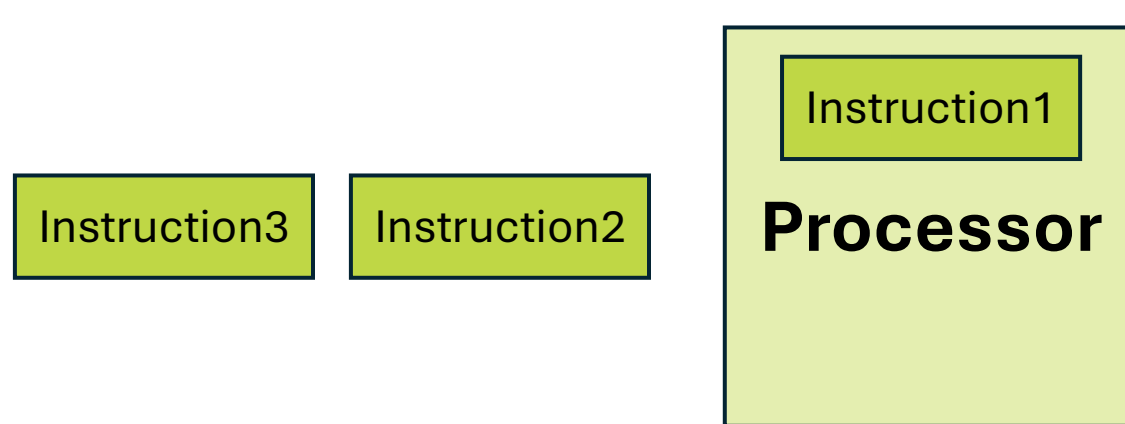
- Sequential processing means that a **CPU executes instructions one after another**, in a **step-by-step** manner, following a strict order. This is the **fundamental way CPUs process data**, ensuring that each task is completed before moving on to the next.



MOV AX, 5 ;
Load the number 5 into register AX

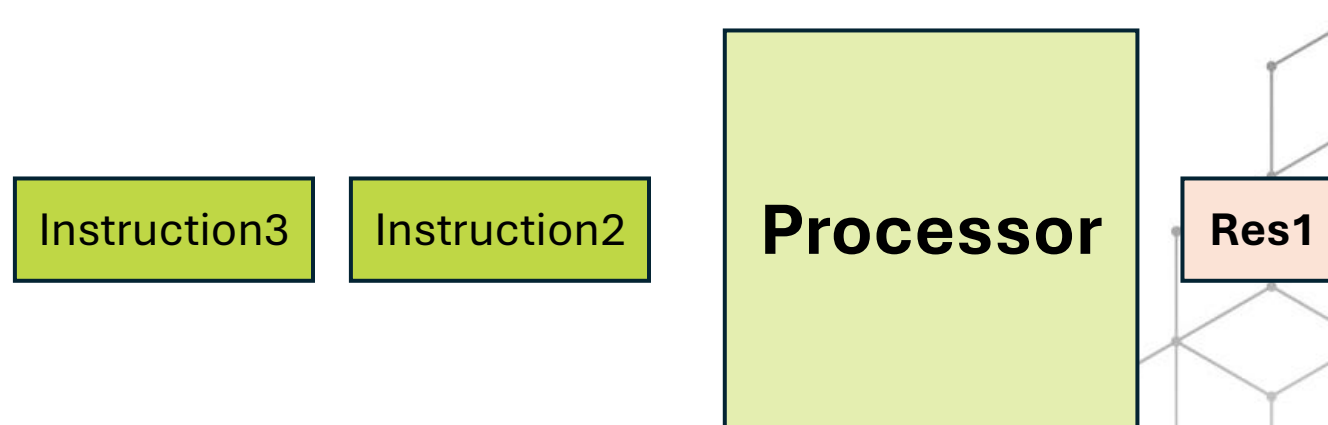
Sequential Processing

- Sequential processing means that a **CPU executes instructions one after another**, in a **step-by-step** manner, following a strict order. This is the **fundamental way CPUs process data**, ensuring that each task is completed before moving on to the next.



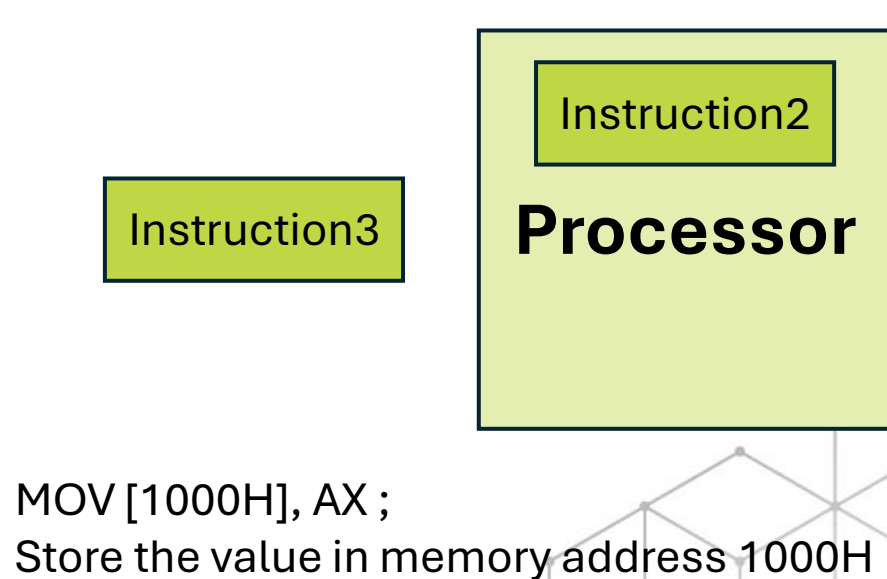
Sequential Processing

- Sequential processing means that a **CPU executes instructions one after another**, in a **step-by-step** manner, following a strict order. This is the **fundamental way CPUs process data**, ensuring that each task is completed before moving on to the next.



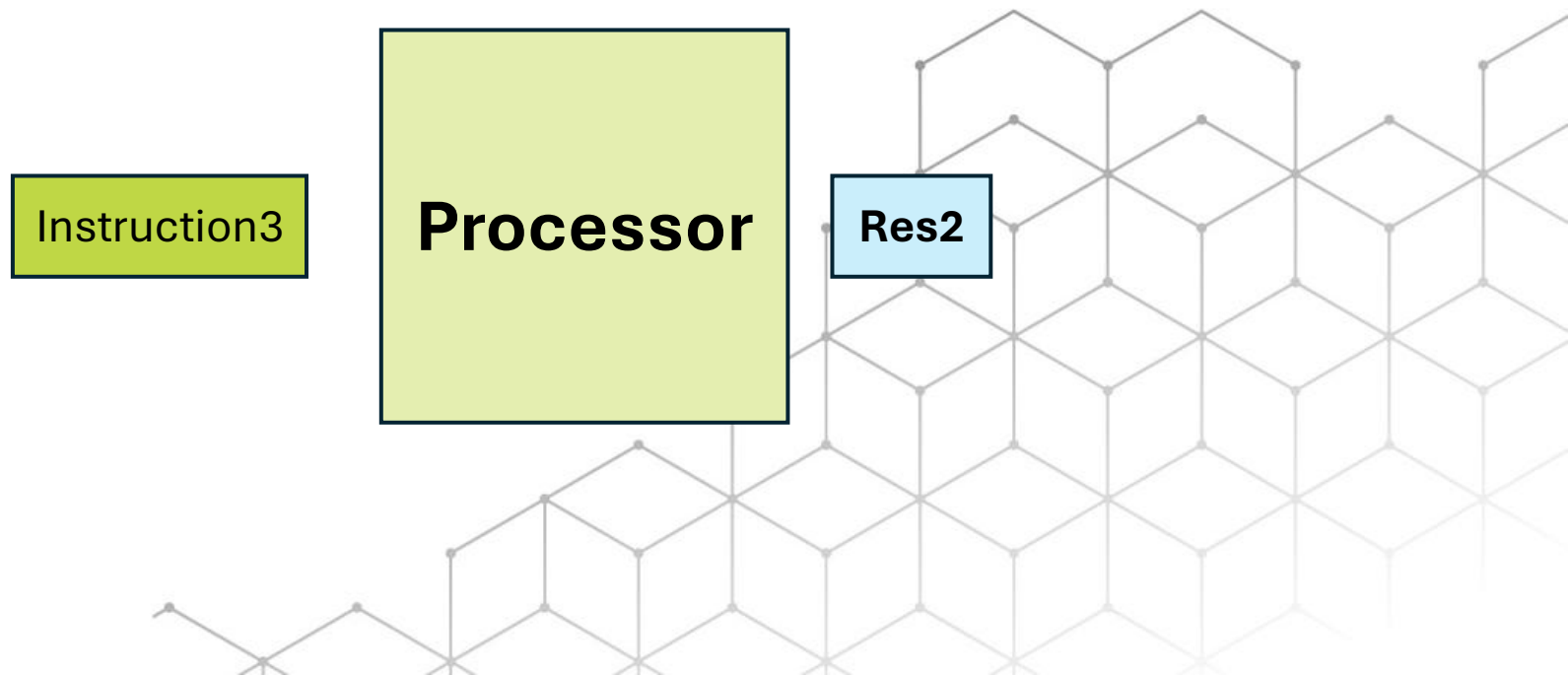
Sequential Processing

- Sequential processing means that a **CPU executes instructions one after another**, in a **step-by-step** manner, following a strict order. This is the **fundamental way CPUs process data**, ensuring that each task is completed before moving on to the next.



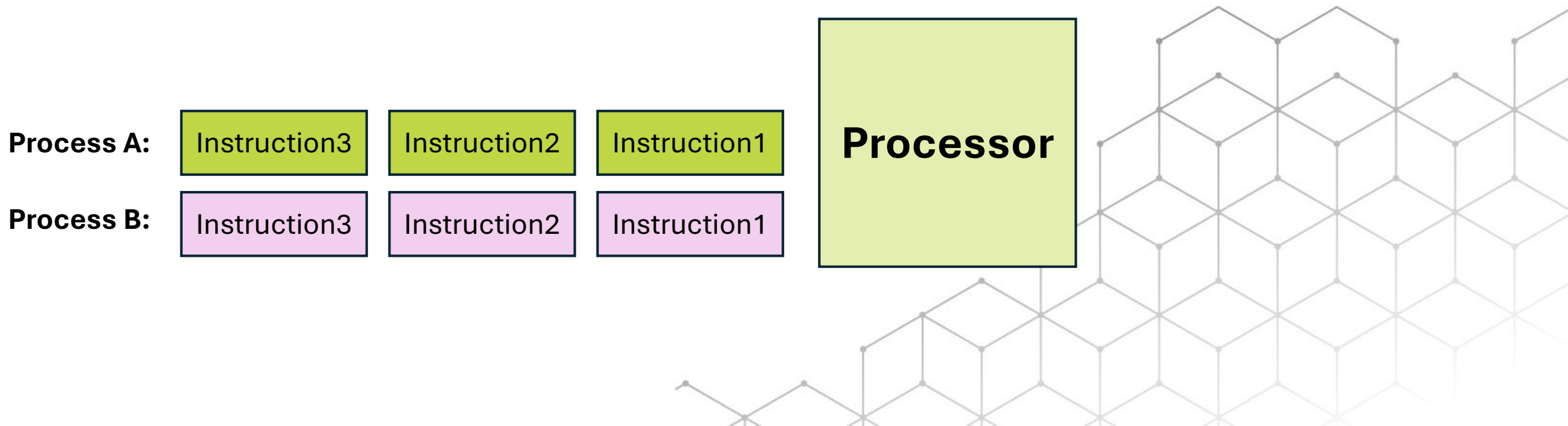
Sequential Processing

- Sequential processing means that a **CPU executes instructions one after another**, in a **step-by-step** manner, following a strict order. This is the **fundamental way CPUs process data**, ensuring that each task is completed before moving on to the next.



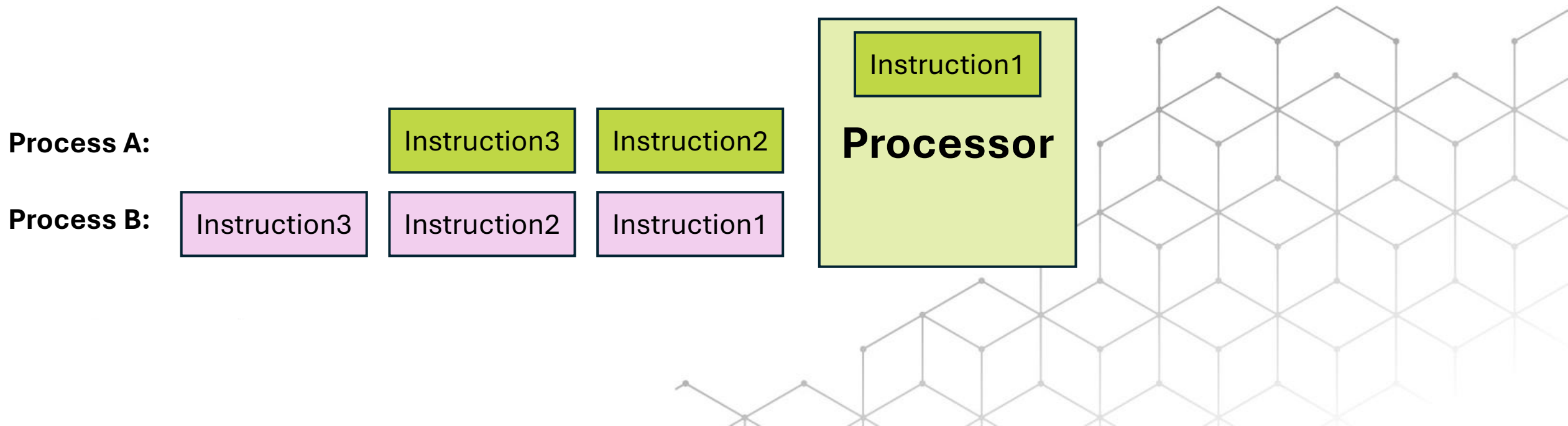
How can it Run Programs in Parallel?

- **Pseudo Parellellism** – Allowing the CPU to work on multiple processes and inner threads (small tasks) at once.



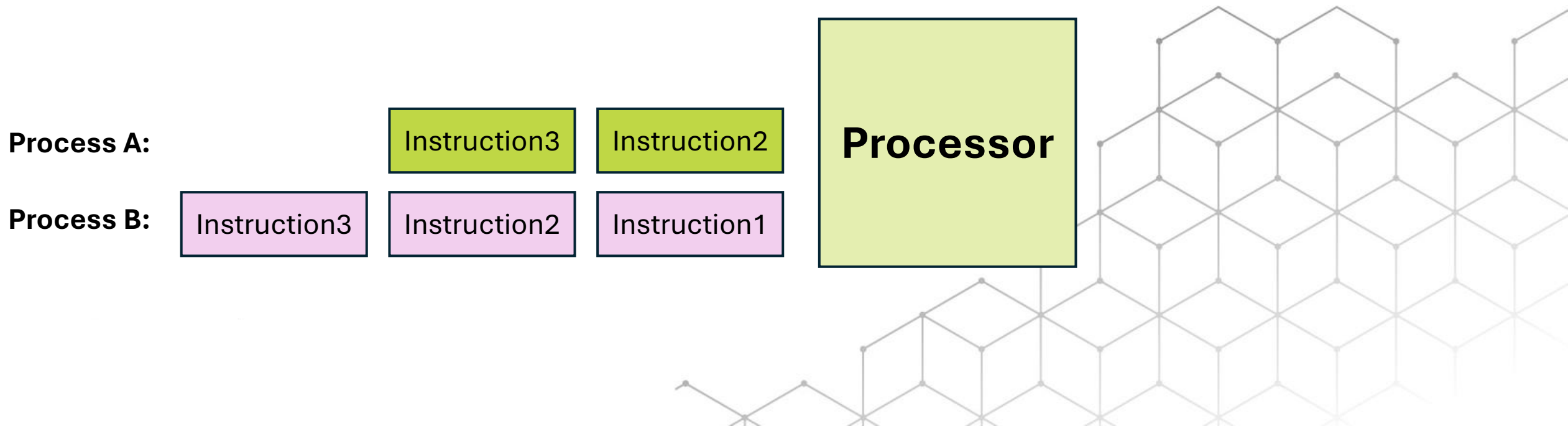
How can it Run Programs in Parallel?

- **Pseudo Parellellism** – Allowing the CPU to work on multiple processes and inner threads (small tasks) at once.



How can it Run Programs in Parallel?

- **Pseudo Parellellism** – Allowing the CPU to work on multiple processes and inner threads (small tasks) at once.



How can it Run Programs in Parallel?

- **Pseudo Parellellism** – Allowing the CPU to work on multiple processes and inner threads (small tasks) at once.

Process A:

Instruction3

Instruction2

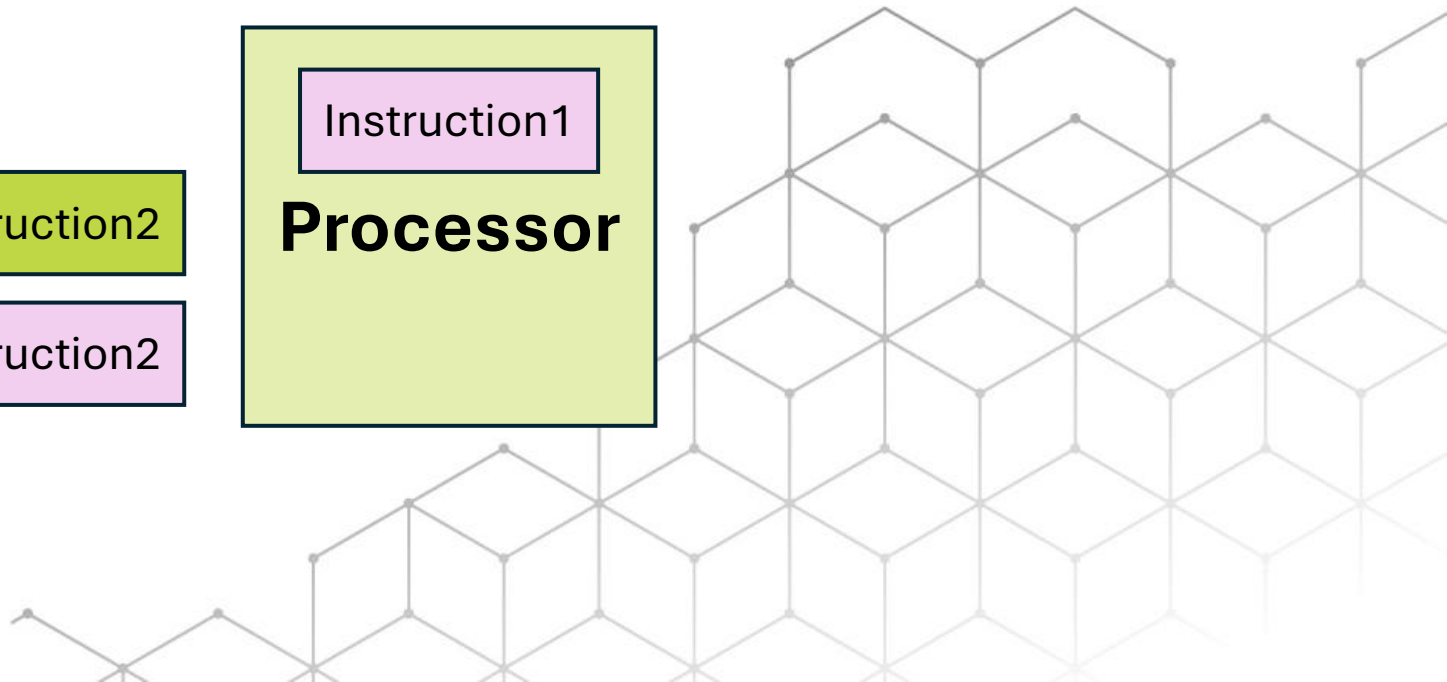
Process B:

Instruction3

Instruction2

Instruction1

Processor



How can it Run Programs in Parallel?

- **Pseudo Parellellism** – Allowing the CPU to work on multiple processes and inner threads (small tasks) at once.

Process A:

Instruction3

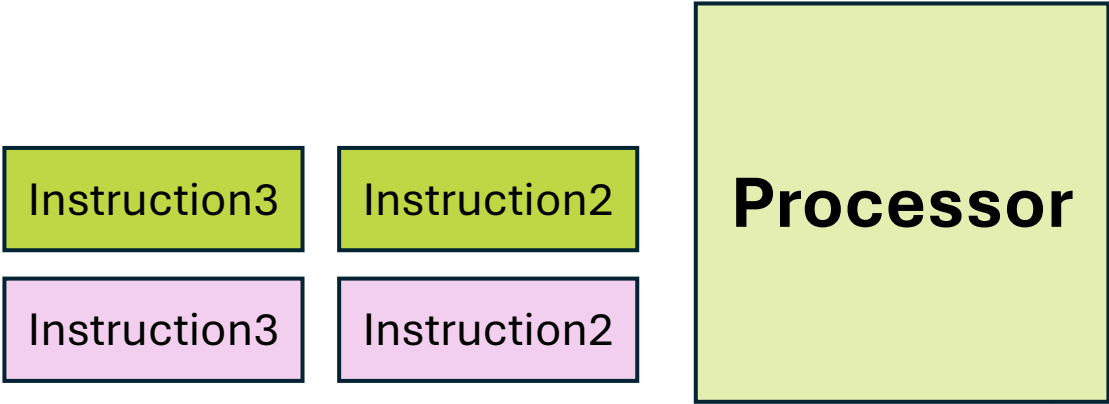
Instruction2

Process B:

Instruction3

Instruction2

Processor

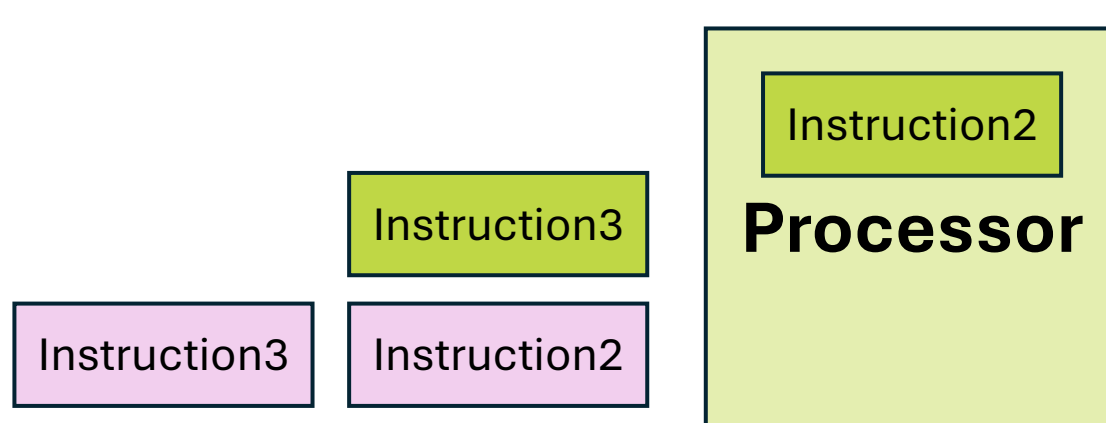


How can it Run Programs in Parallel?

- **Pseudo Parellellism** – Allowing the CPU to work on multiple processes and inner threads (small tasks) at once.

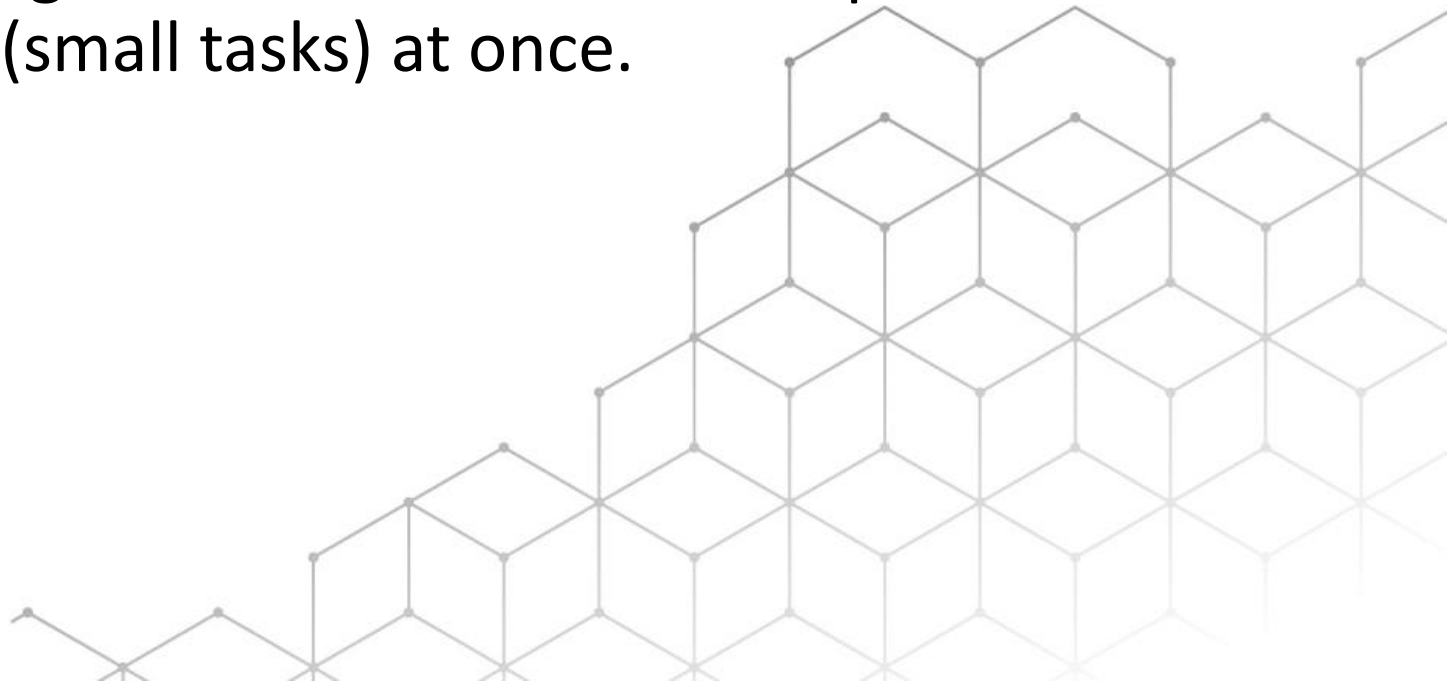
Process A:

Process B:



How can it Run Programs in Parallel?

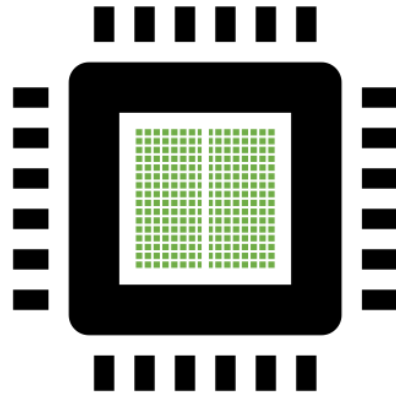
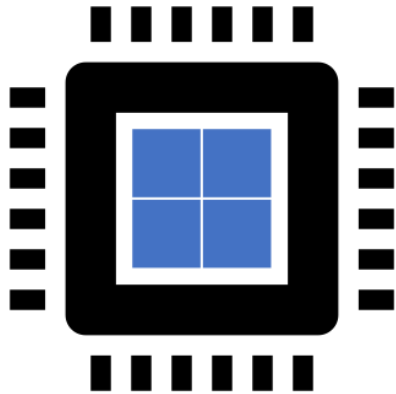
- Although CPUs are **sequential**, they use techniques:
- **Multiple Cores** – Modern CPUs have multiple cores, allowing some degree of **parallel execution** for multitasking.
- **Pseudo Parallelism** – Allowing the CPU to work on multiple processes and inner threads (small tasks) at once.



CPU vs Human brain

Feature	CPU (Computer Processor)	Human Brain
Processing Style	Serial Processing (mostly one task at a time)	Massively Parallel Processing (billions of neurons firing at once)
Speed	Executes billions of instructions per second (GHz speed)	Billions of neurons firing
Energy Efficiency	Consumes dozens to hundreds of watts	Consumes ~ 20 watts (super efficient)
Memory Storage	Uses RAM & Disk Storage	Stores info in neural connections (synapses)
Learning Ability	Needs explicit programming	Learns from experience (plasticity)
Error Handling	Crashes or produces errors on bad inputs	Adapts & corrects for errors (intuitive reasoning)

CPU vs GPU: Different Strengths



CPU	GPU
Central Processing Unit	Graphics Processing Unit
4-8 Cores	100s or 1000s of Cores
Low Latency	High Throughput
Good for Serial Processing	Good for Parallel Processing
Quickly Process Tasks That Require Interactivity	Breaks Jobs Into Separate Tasks To Process Simultaneously
Traditional Programming Are Written For CPU Sequential Execution	Requires Additional Software To Convert CPU Functions to GPU Functions for Parallel Execution



Chef Metaphor Again

- Think of a CPU as a single master chef—skilled at making complex, custom dishes one at a time.
- Meanwhile, a GPU is like a fast-food kitchen, with many workers making thousands of burgers in parallel.
- A CPU is best for **sequential**, complex tasks that require flexibility.
- A GPU is best for **parallel workloads** where thousands of simple calculations need to be done simultaneously.

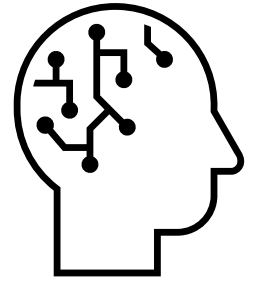


Graphical Processing Units

- Gamers love **GPUs** because they handle **real-time rendering** of computer graphics.
 1. Render (show) **millions of pixels per sec**, **fast parallel computations**.
 2. Multiple tasks at once (**shading, lighting, physics, textures, etc.**).
 3. Gameplay at **60-240 FPS** for fast reactions.
 4. Without a GPU, games stutter or lag.



Deep Learning Models & GPU



- AI researchers use **GPUs** because DL requires **massive computations**.

1.Parallelism for Matrix Operations

1. Neural networks rely on **matrix multiplications**, which GPUs execute in parallel.
2. AI training on a CPU could take **weeks**, while a GPU can reduce it to **days or hours**.

2.Tensor Cores & AI-Specific Hardware

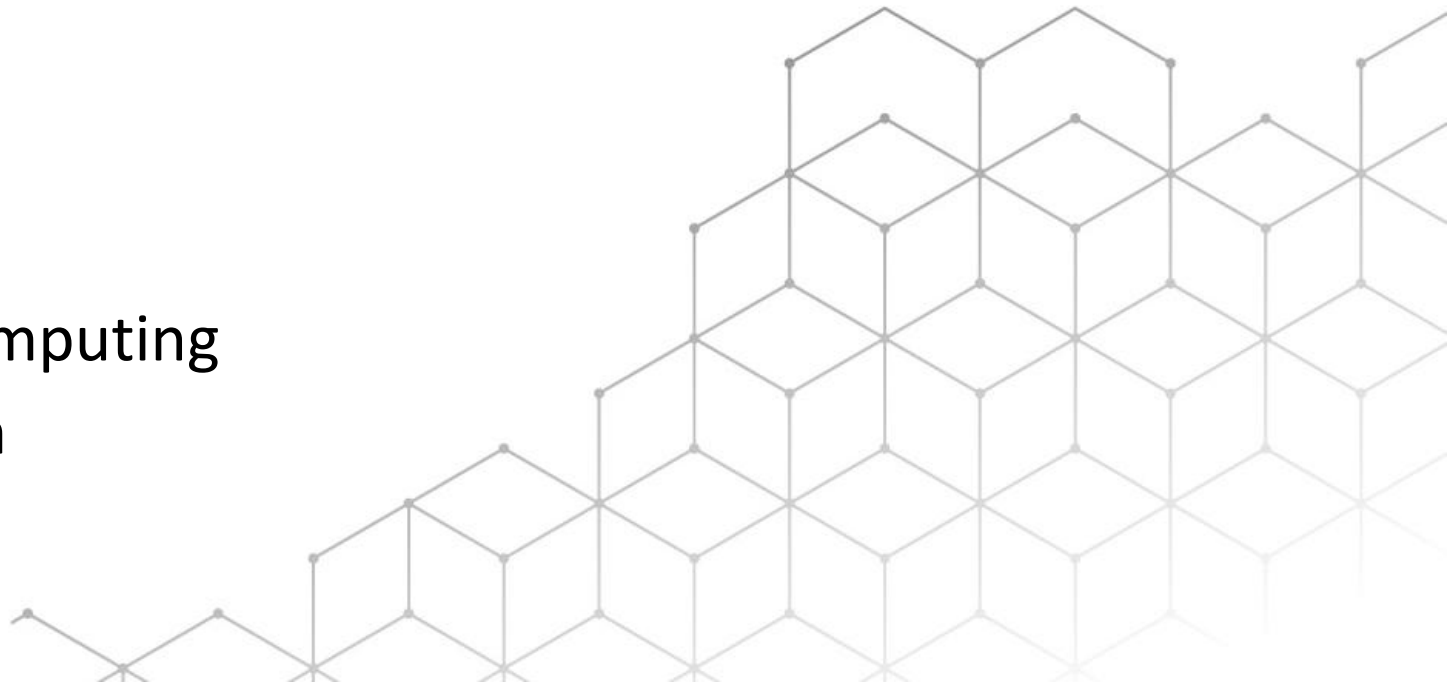
1. **NVIDIA GPUs (like the A100, H100, and RTX 4090)** include **Tensor Cores** (20x).
2. Tensor Cores can **automatically adjust precision**.

3.Faster Model Inference (Real-Time AI)

1. **NVIDIA's H100 GPUs** are for **language models (Transformer Engine Support)**

Agenda for Today...

- Computing Resources
- Role of the Processing Unit (CPU & GPU)
- Primary (RAM) and Secondary Storage (Disk)
 - Disk is not always the bottleneck
 - RAM or Storage?
 - How RAM works
- Computational Scalability
- Distributed Computing
- Components of Distributed Computing
- Computing Resources in Action



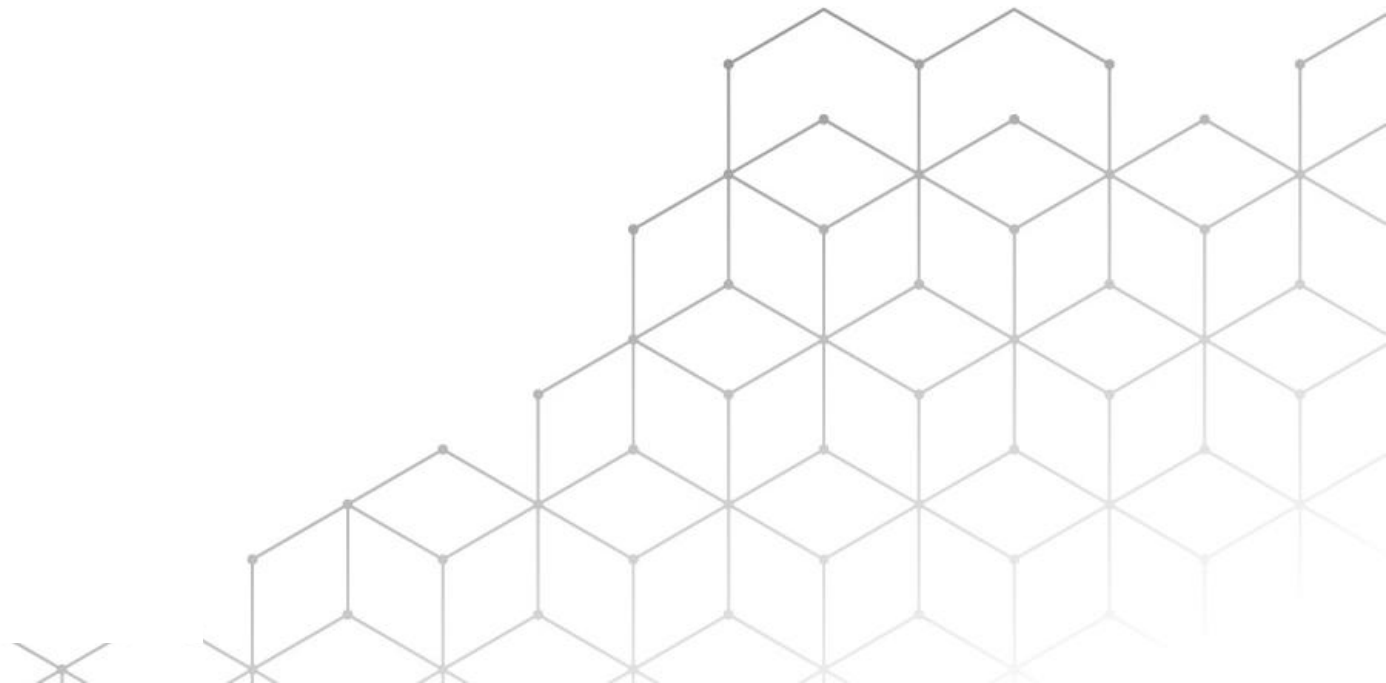
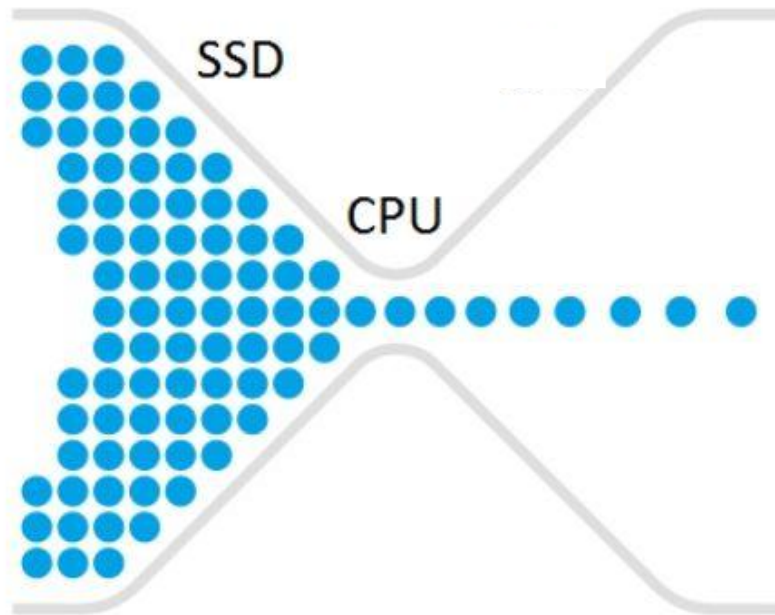
Questions

- Why do computers slow down when running too many programs, even if they have large storage capacity?
- If you had to remove either RAM or storage from your laptop, which one would you choose and why?
- Why do video editors and gamers need more RAM compared to casual web users?



Bottleneck

- Even if your computer has **storage capacity (HDD/SSD)**, it can still slow down when too many programs are running. That's because **storage is not the main factor affecting speed**— the **CPU, RAM, Cache** play a bigger role.



RAM or Storage

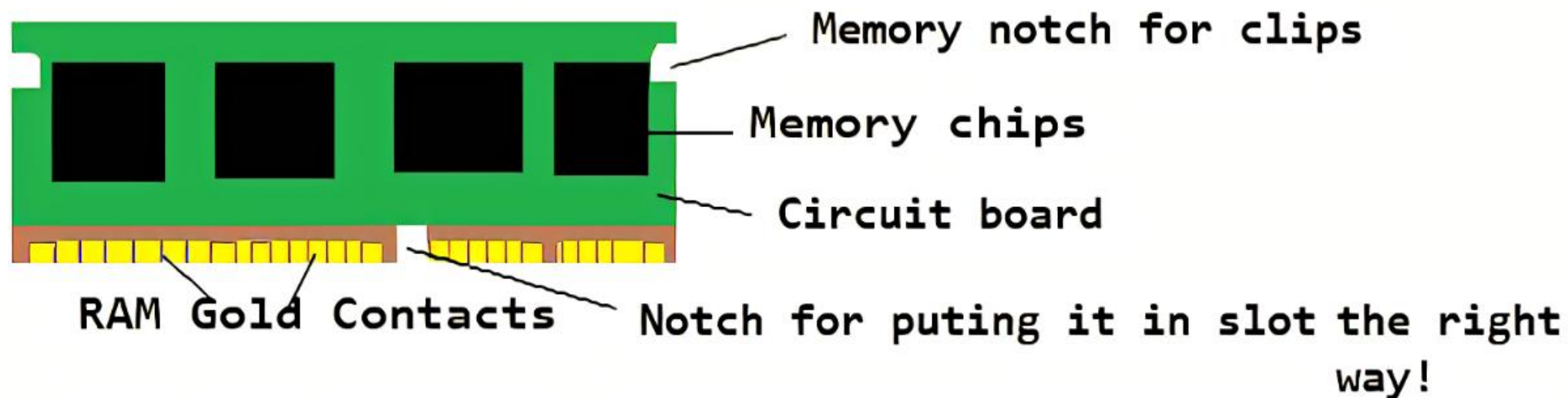
- RAM is Essential for Running Programs
 - Without RAM, the operating system (OS) won't even boot.
 - The CPU needs RAM to store active processes—without it, the computer is unusable.
- Storage Can Be Replaced Temporarily:
 - If storage is removed, I could still:
 - Boot from an external USB drive, SSD, or cloud storage.



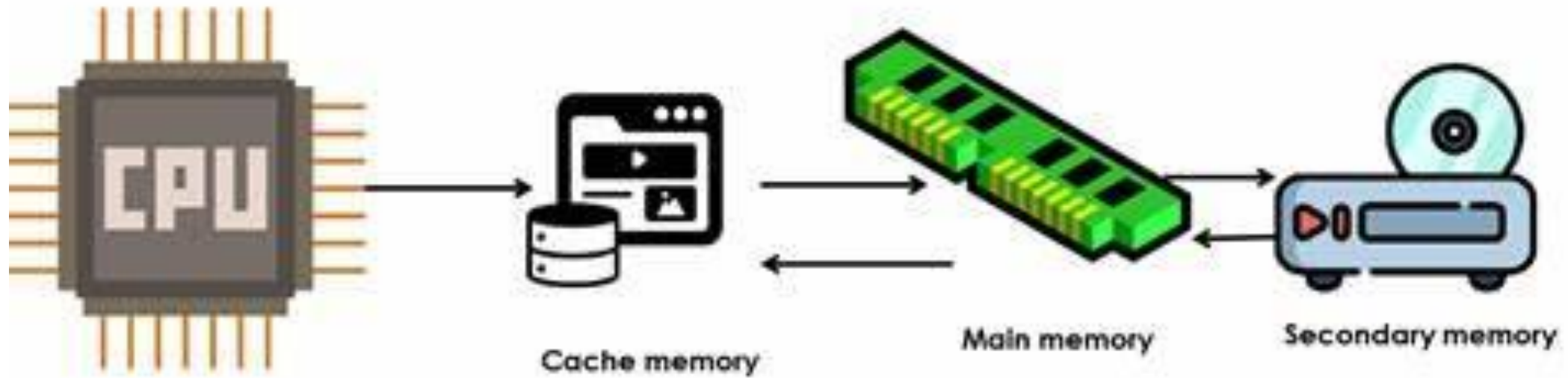
How RAM works

- RAM (**Random Access Memory**) is a **temporary storage space** where a computer keeps **active** data for quick access.
- If RAM is insufficient, the system **swaps data to disk**, causing **lags and slow performance**.

Memory/RAM Diagram

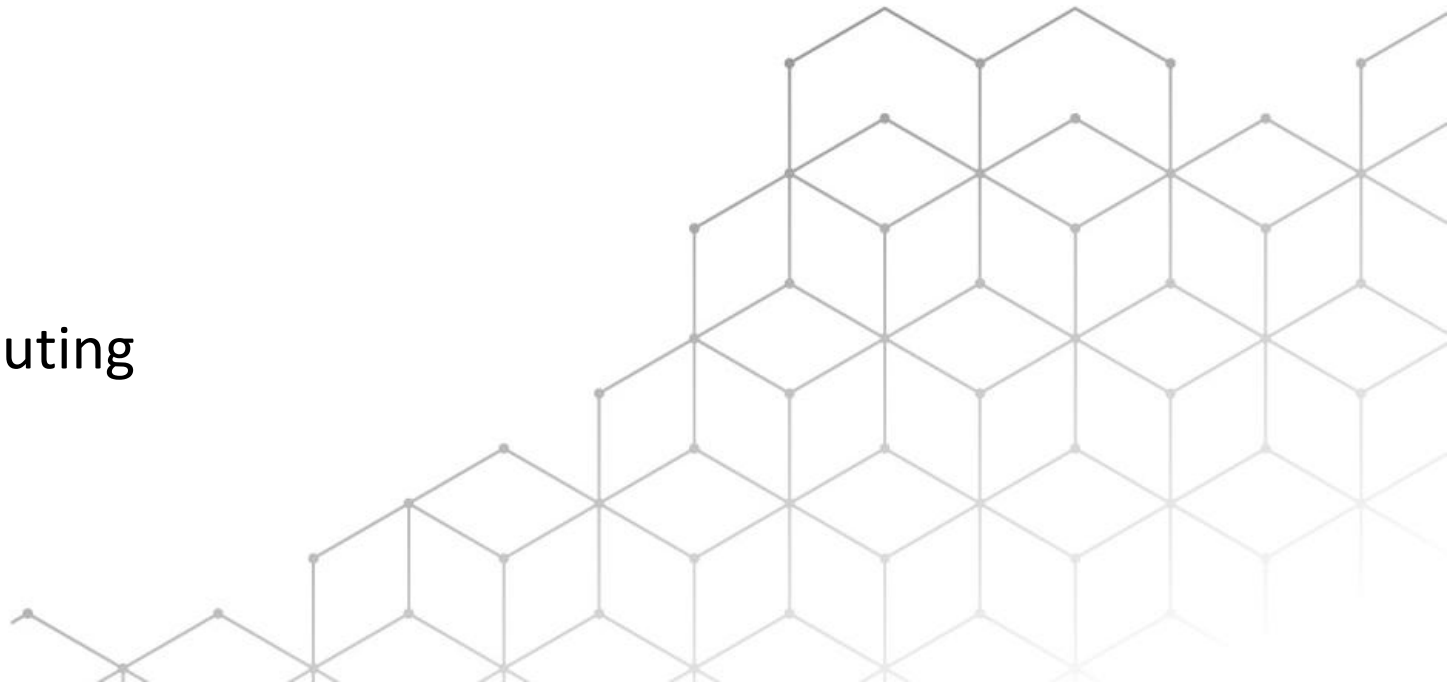


Cache



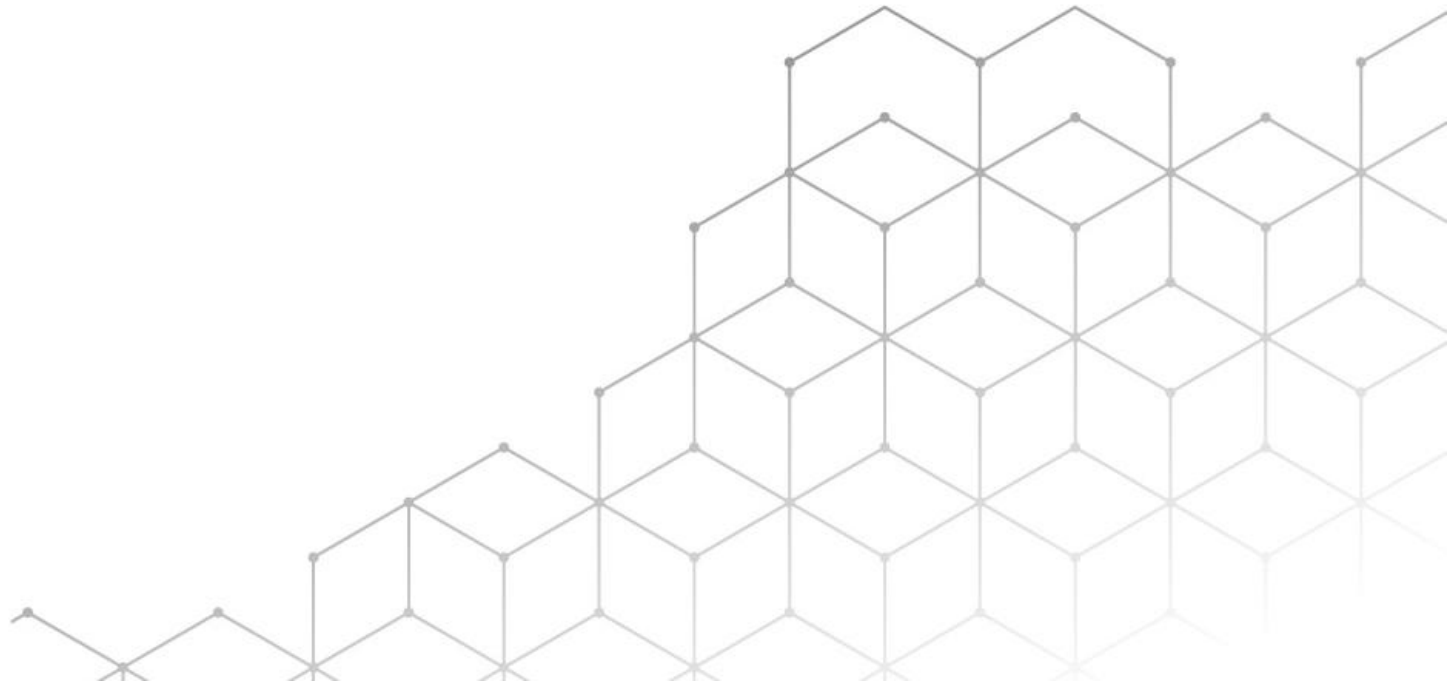
Agenda for Today...

- Computing Resources
- Role of the Processing Unit (CPU & GPU)
- Primary (RAM) and Secondary Storage (Disk)
- **Computational Scalability**
 - What is computational scalability?
 - How to achieve scalability?
 - Scale up (Vertical)
 - Scale up Architectures
 - Scale out (Horizontal)
- Distributed Computing
- Components of Distributed Computing
- Computing Resources in Action



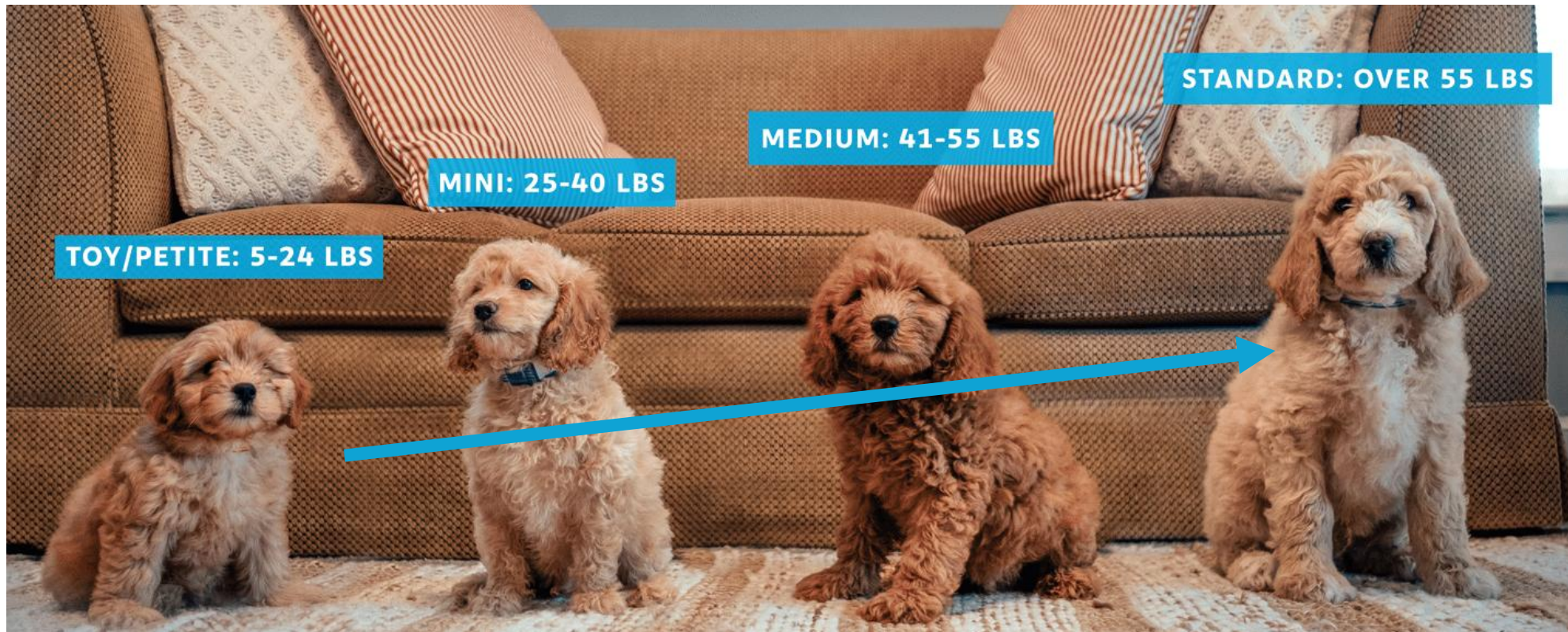
Definition of Scalability

- The ability of a system to grow larger
- To achieve **scalability** to accommodate big data we can:
 - Scale up: Add storage (RAM, disk) or processor to a single machine
 - Scale out: Add machines to the network



Scale up

- Add storage (RAM, disk) or processor to a single machine



NOVA

IMS

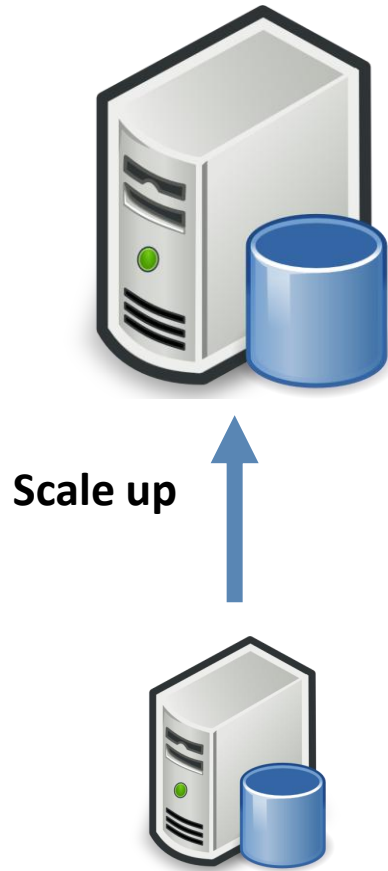
Information
Management
School

Scale Out

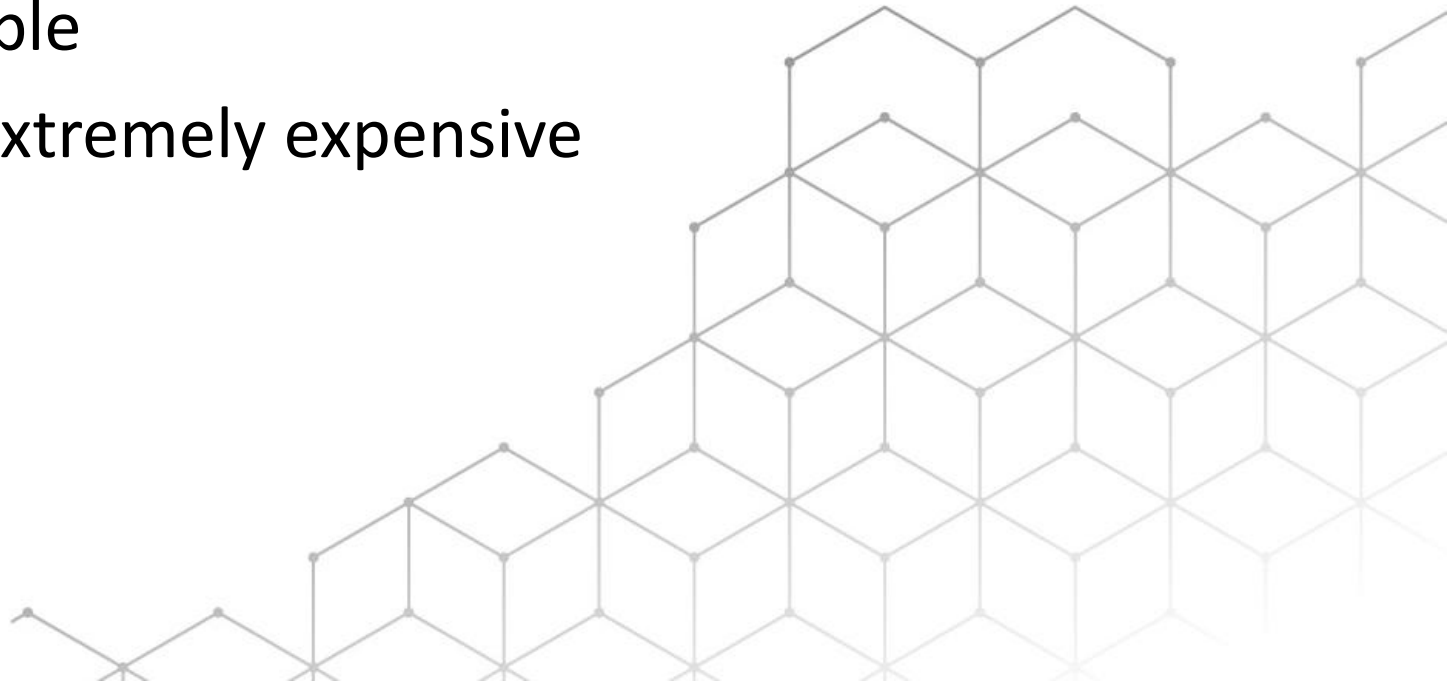
Scale out: Add machines to the network



Scale up (Vertical Scaling)

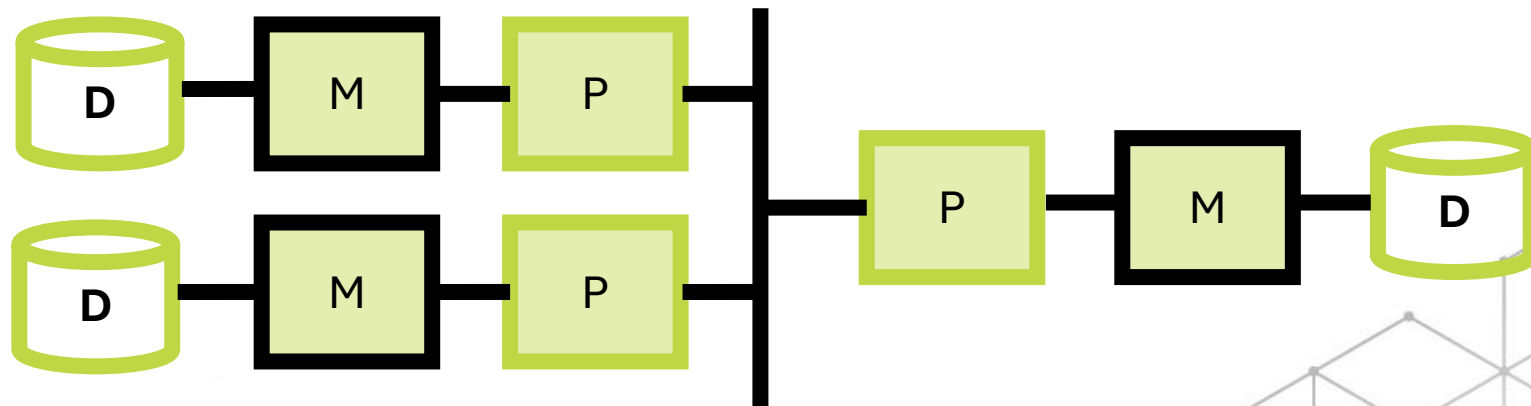


- Move to a bigger box:
 - Better CPU or cores
 - Better RAM memory
 - Better disk
- Reliable
- But extremely expensive



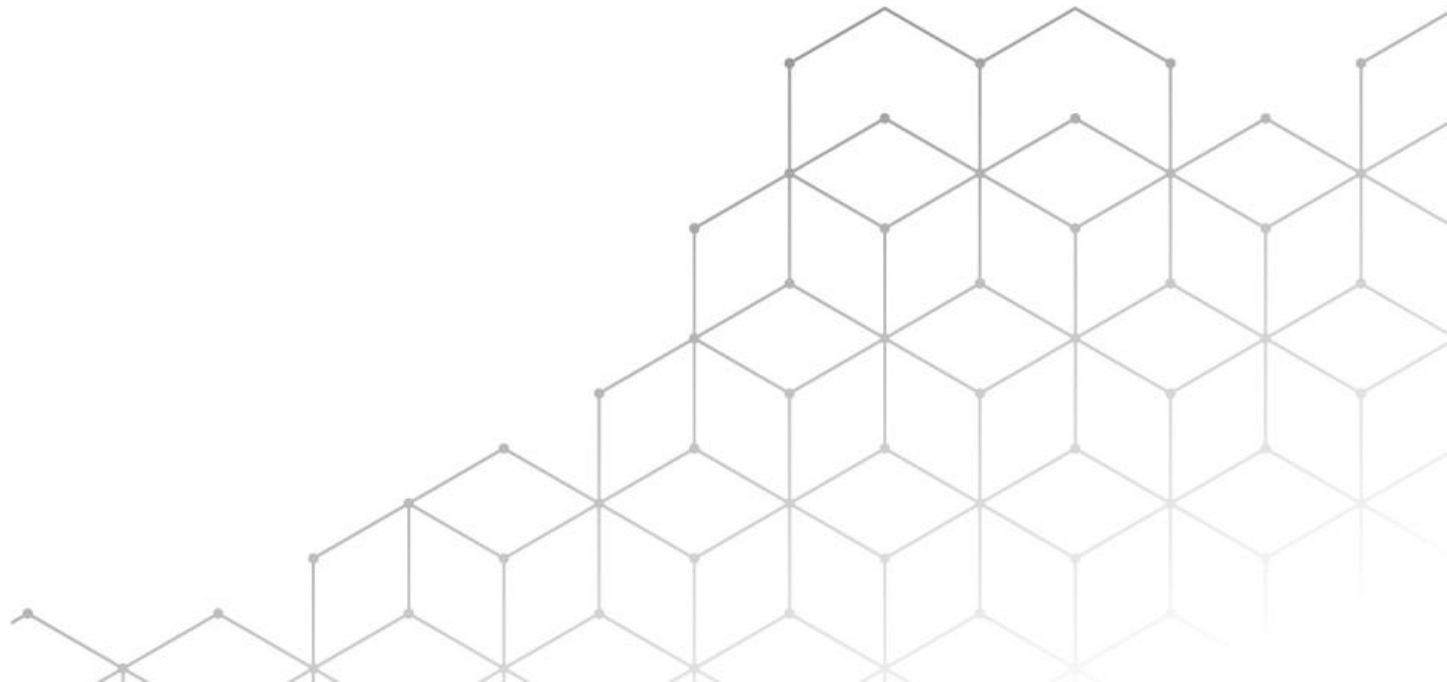
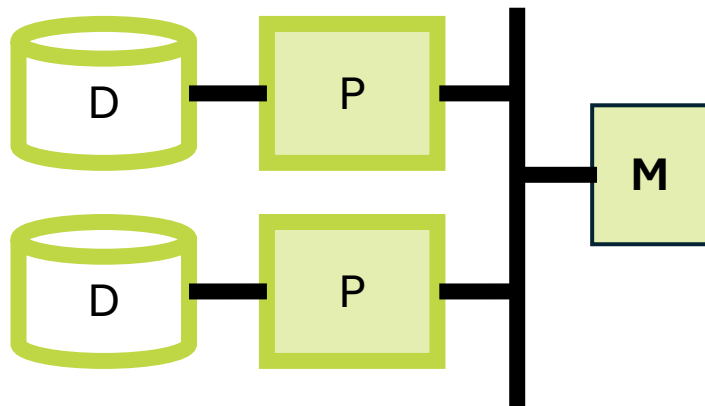
Architecture of Shared Nothing

- Processors share neither a memory nor disk.
- Shared-nothing multiprocessors can be scaled up to thousands of processors without interference.



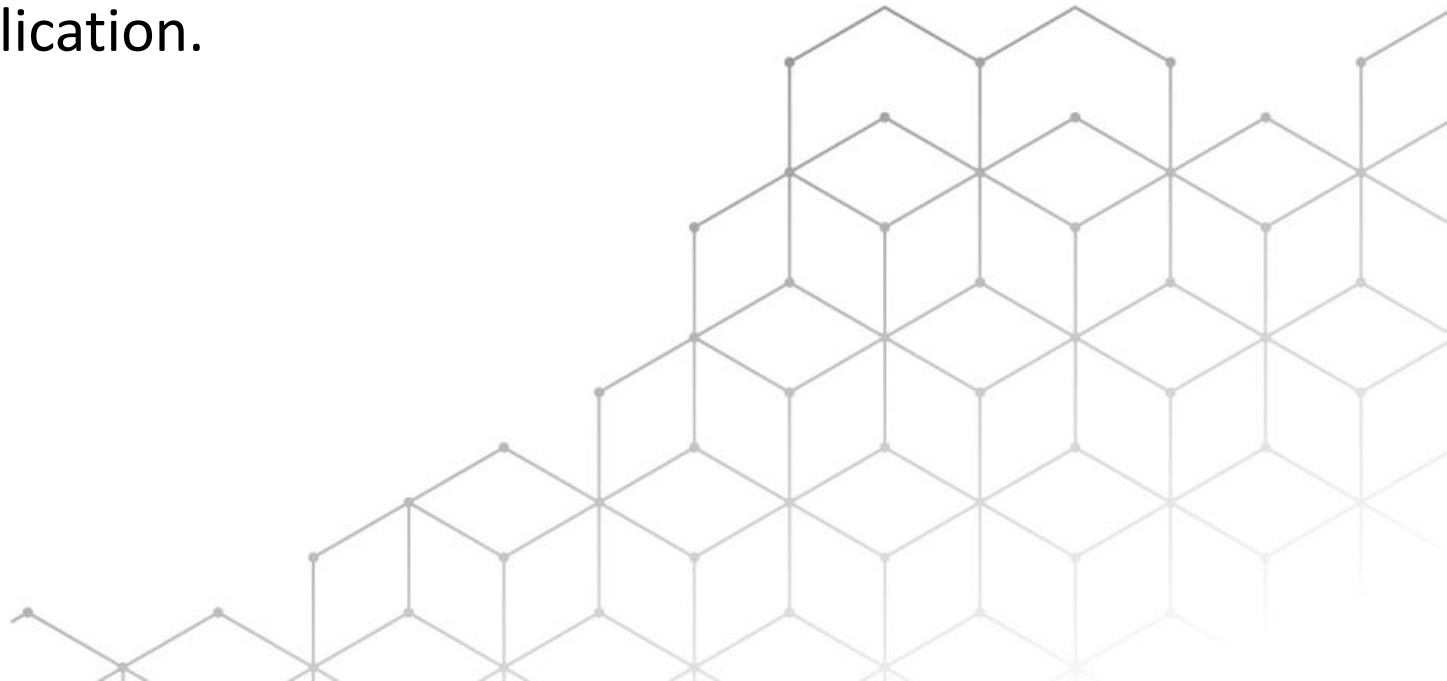
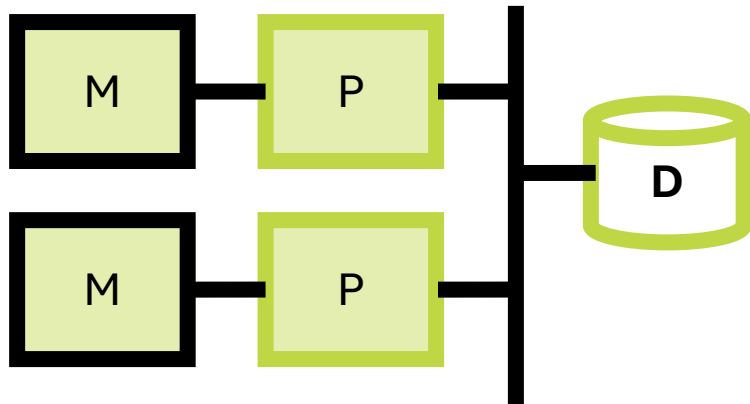
Architecture of Shared Memory

- Multiple processors share the same memory space, making communication between them faster and more efficient.
 - This works well within a **single machine** but does not scale efficiently across multiple nodes (distributed systems).



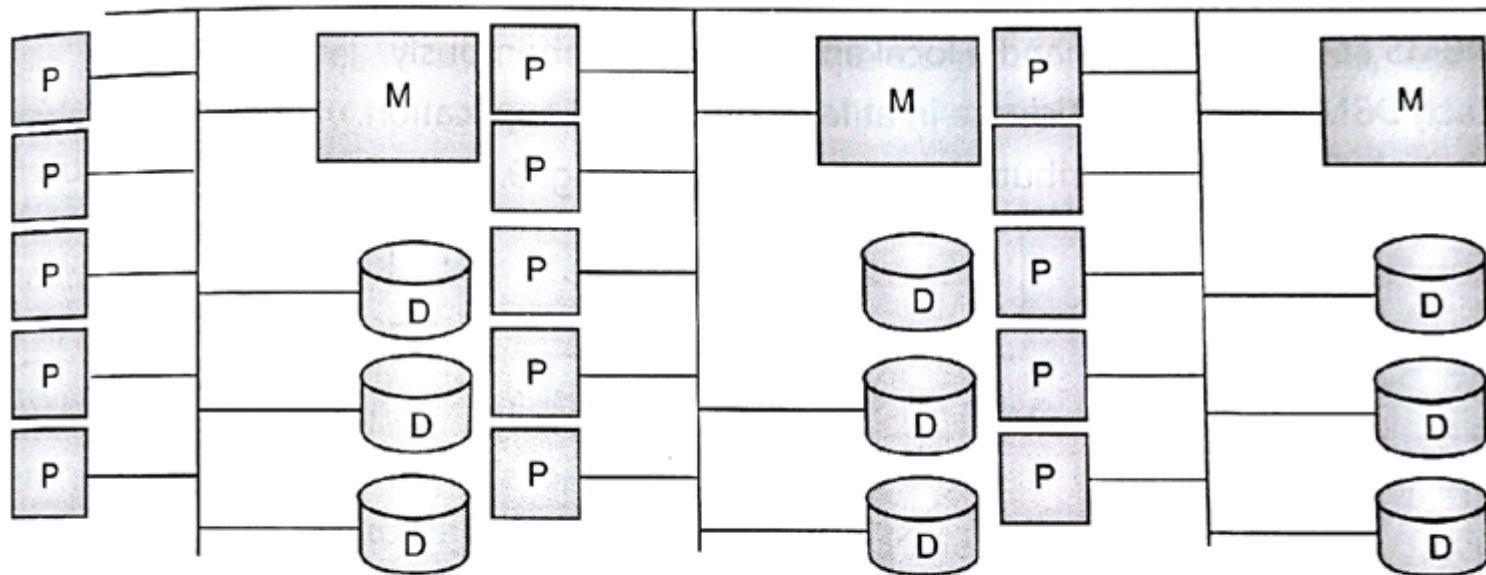
Architecture of Shared Disk

- In traditional **scale-up systems**, multiple processors (CPUs) share access to a single storage system.
- **Advantages:**
 - Single, consistent view of data across all processors.
 - No need for complex data replication.



Hybrid (Hierarchical) Architecture

- Combines characteristics of shared-memory, shared-disk, and shared-nothing architectures.

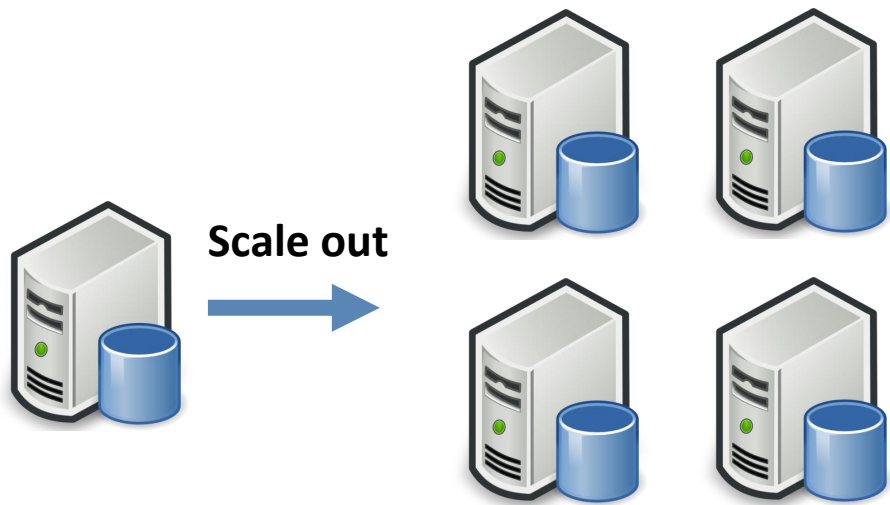


P → Processor

M → Memory

D → Disk

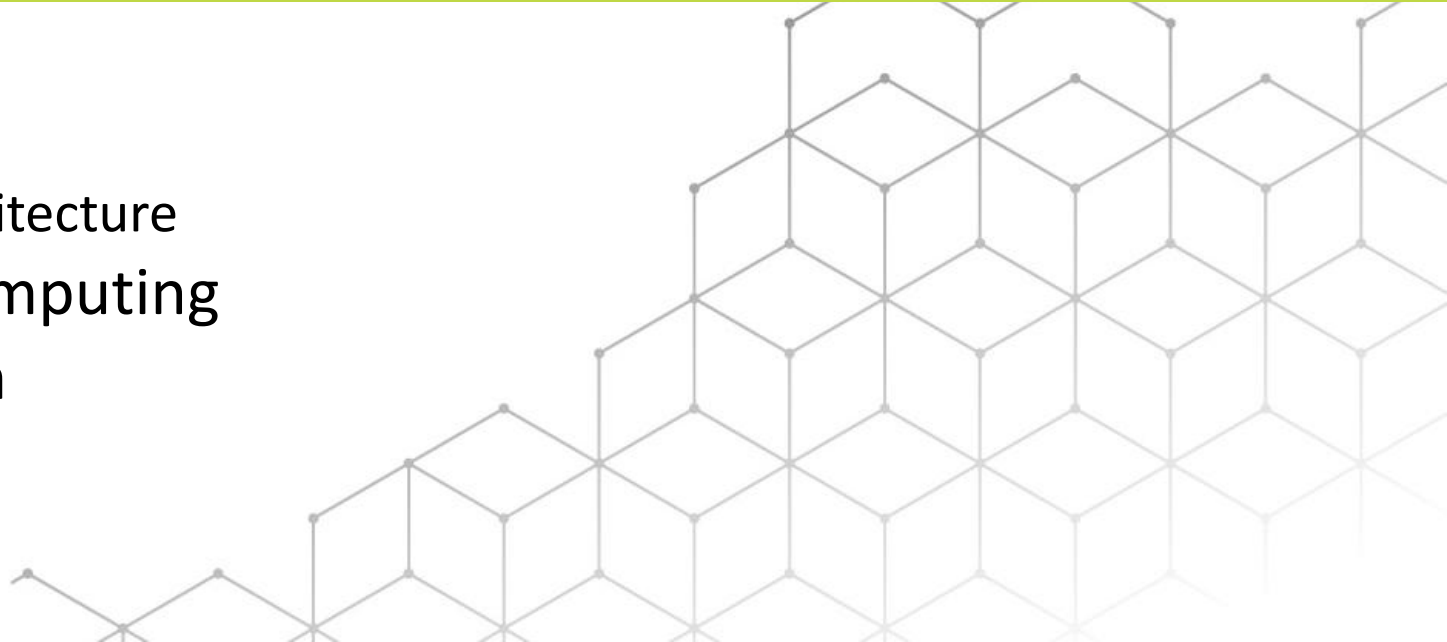
Scale out (Horizontal Scaling)



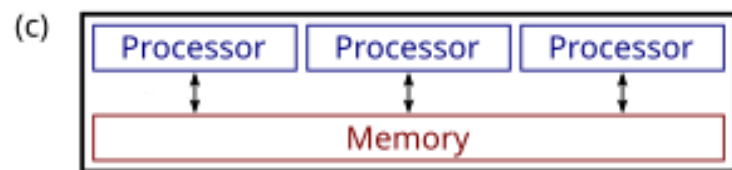
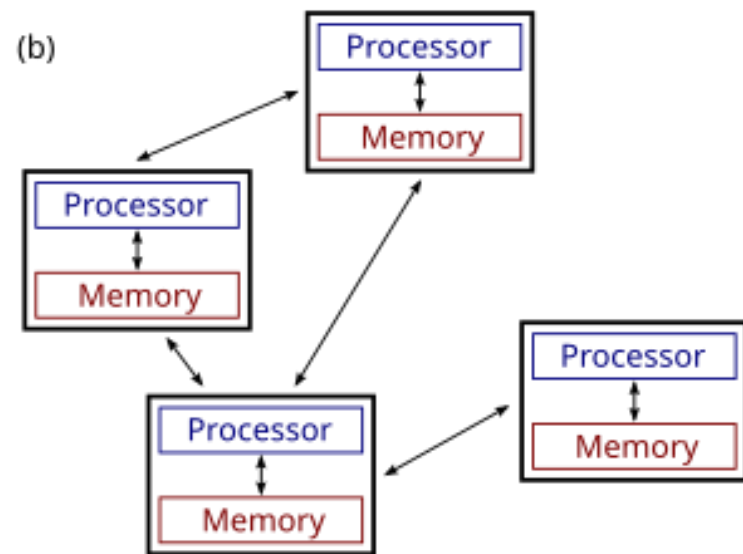
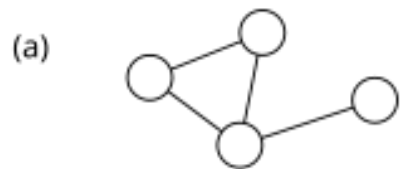
- Distribute the data & processing across multiple nodes.
- Slower due to operational overheads
- Architecture-aware algorithm design
- But much cheaper

Agenda for Today...

- Computing Resources
- Role of the Processing Unit (CPU & GPU)
- Primary (RAM) and Secondary Storage (Disk)
- Computational Scalability
- Distributed Computing
 - Definition of Distributed System
 - Why Distributed Systems?
 - Basic Distributed Architectures
 - Key Features of Distributed Architecture
- Components of Distributed Computing
- Computing Resources in Action



Definition of Distributed Computing System



- A **Distributed Computing System (DCS)** is a system where multiple independent computing nodes work together as a single unit to perform computations, process data, and manage resources.
- These systems (a) and (b) are designed to **scale out** by adding more nodes rather than scaling up by upgrading a single machine (c).

Properties of Distributed Systems

Multiple Independent Nodes

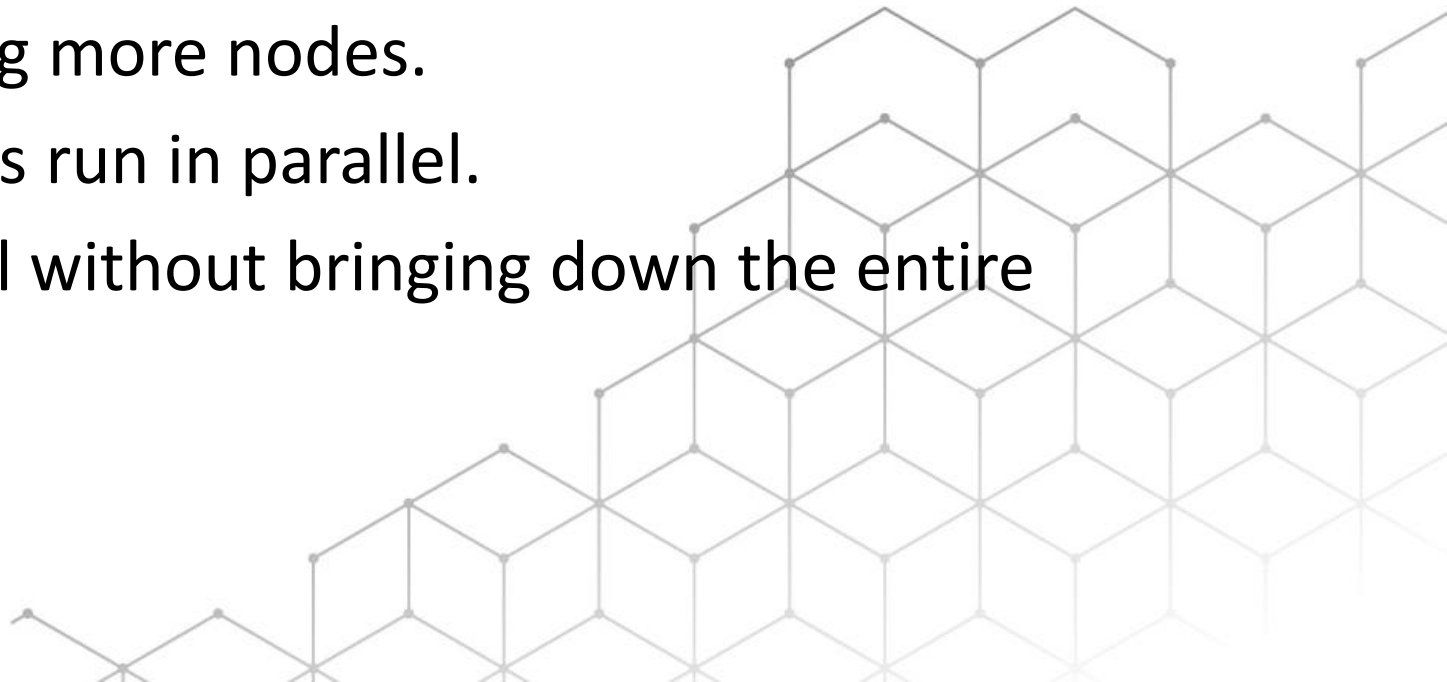
1. Each node (computer) has its own CPU, memory, and storage.
2. Nodes communicate via a network.

Decentralized Processing: Workload is distributed across nodes.

Scalability: Can grow by adding more nodes.

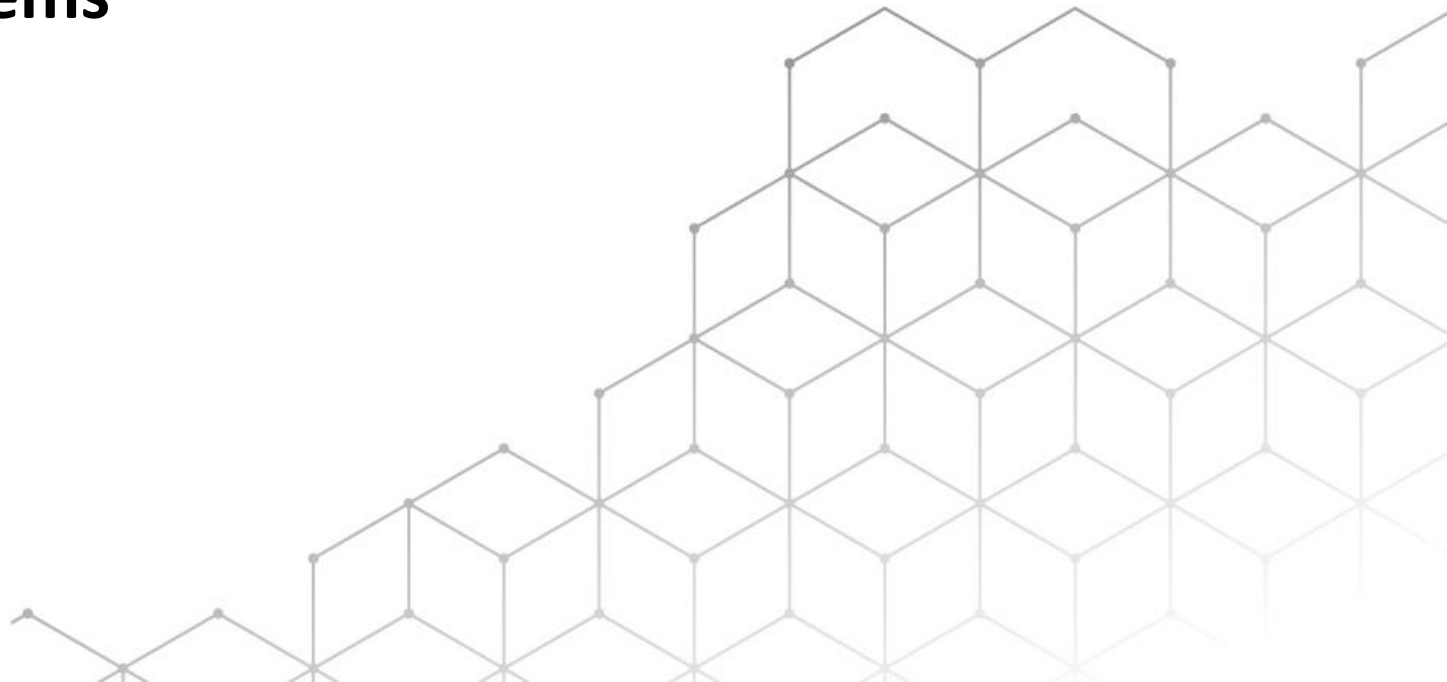
Parallelism: Multiple processes run in parallel.

Fault Tolerance: Nodes can fail without bringing down the entire system.



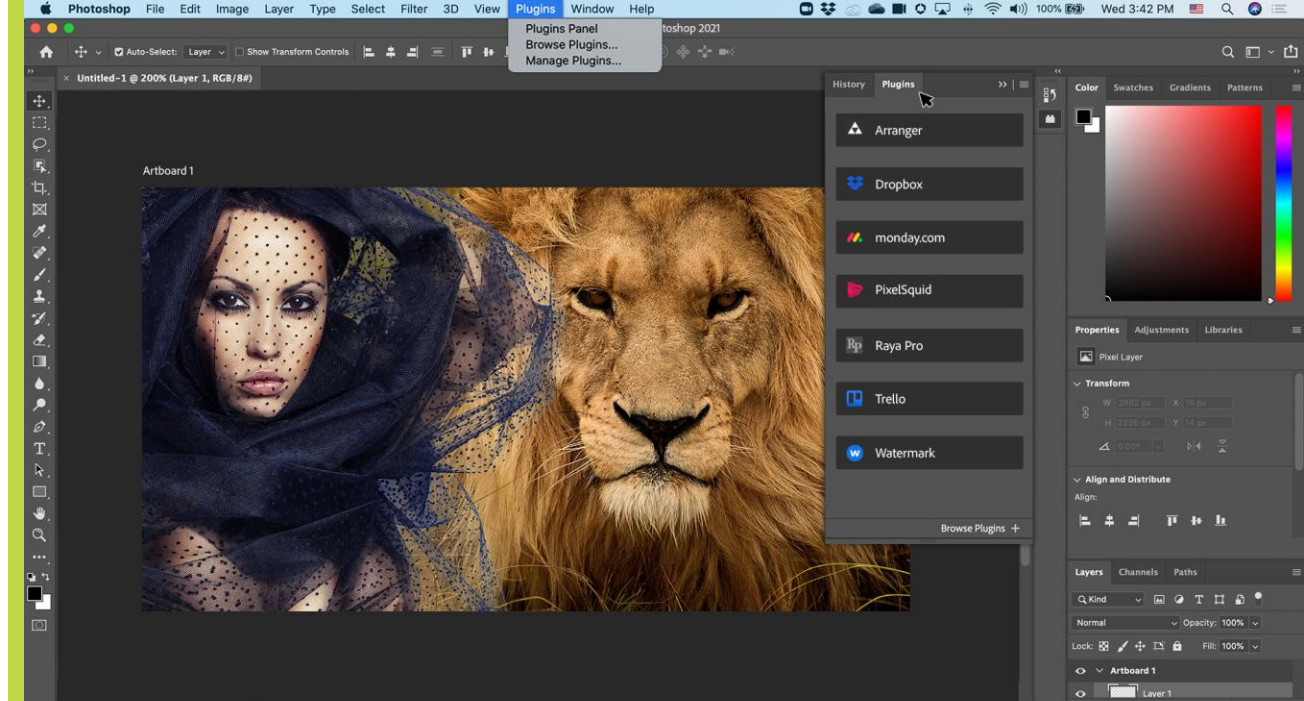
Examples of Distributed Computing Systems

- **Cloud Computing Platforms** (AWS, Google Cloud, Azure)
- **Big Data Platforms** (Apache Hadoop, Apache Spark)
- **Distributed Databases** (Cassandra, MongoDB, Google Spanner)
- **Internet of Things (IoT) Systems**



Examples

- A laptop running Photoshop
→ Traditional computing.
- A data center running AI
training on thousands of
GPUs →
- Distributed computing.



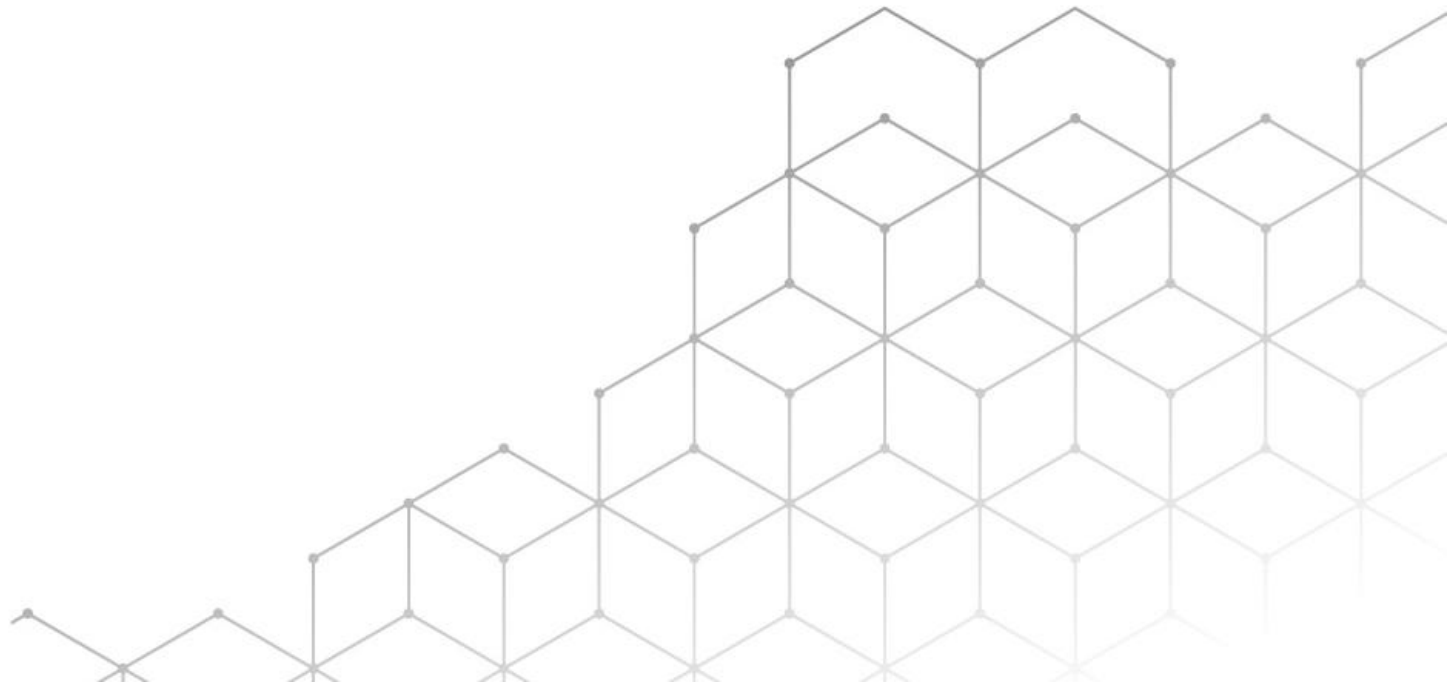
Why Distributed Systems?

Traditional single-server solutions fail at large scales.

- Enable handling large datasets and concurrent requests.

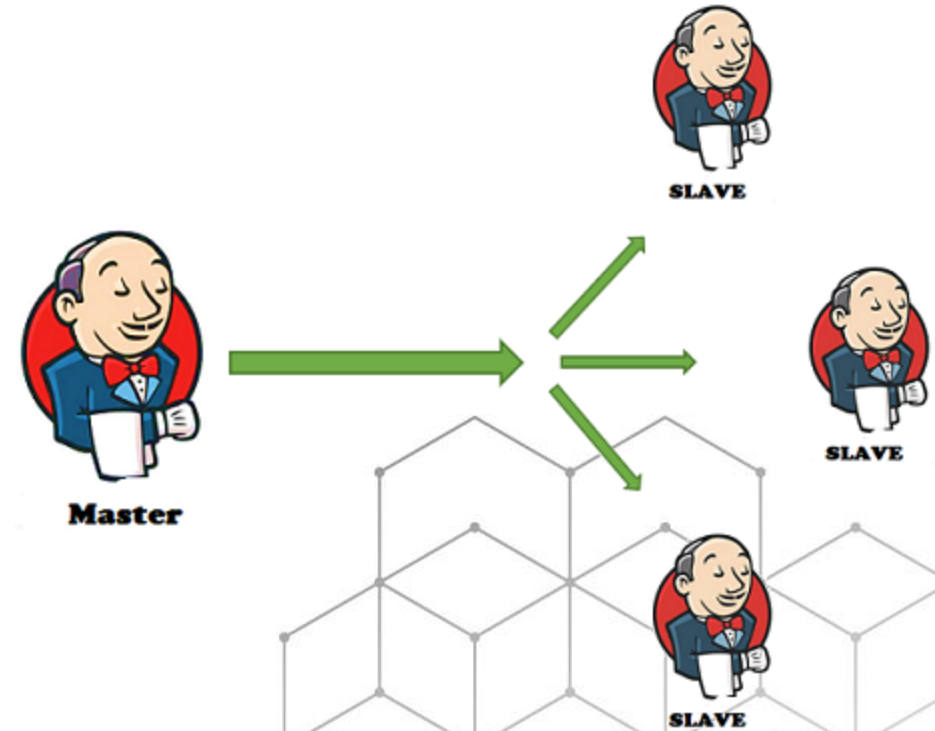
- **Basic Distributed Architectures:**

- Master-Slave Model
- Client-Server Model
- Peer-to-Peer Model



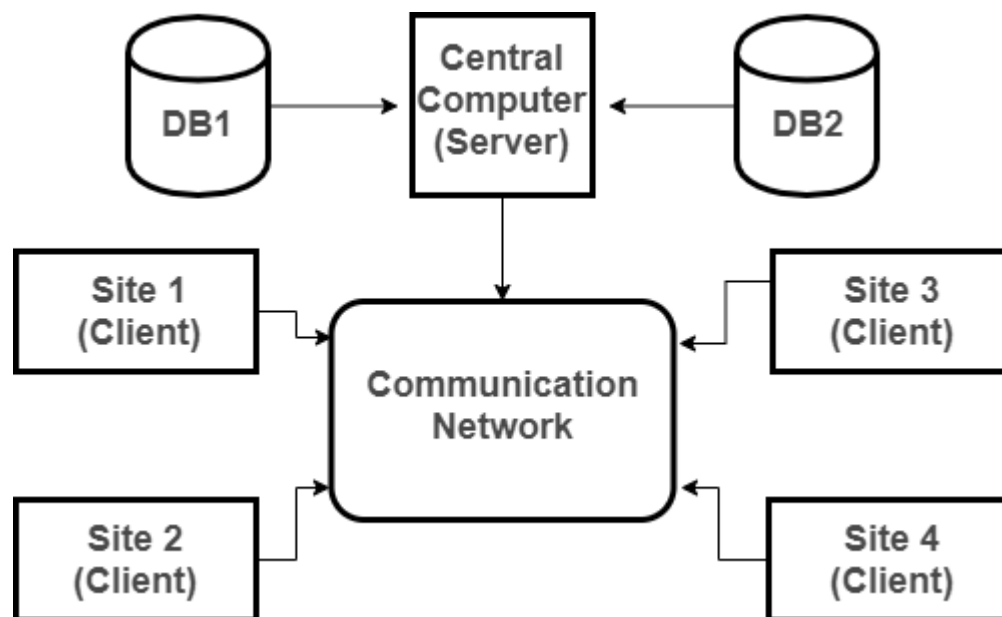
Master-Slave Architecture

- **Definition:** One central node (Master) controls multiple subordinate nodes (Slaves).
- **Control Mechanism:** The **Master** assigns tasks, and **Slaves** execute them.
- **Communication:** Mostly **one-way**; the Master gives instructions, and the Slave responds.



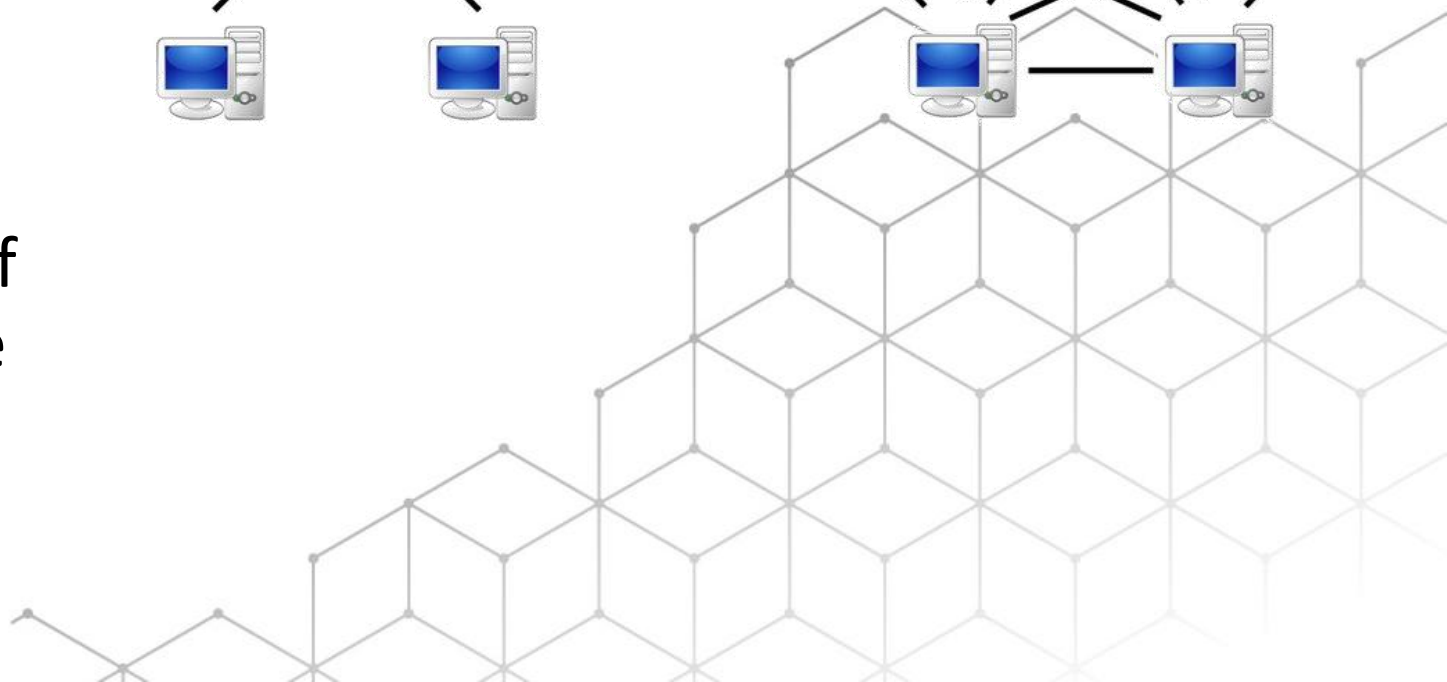
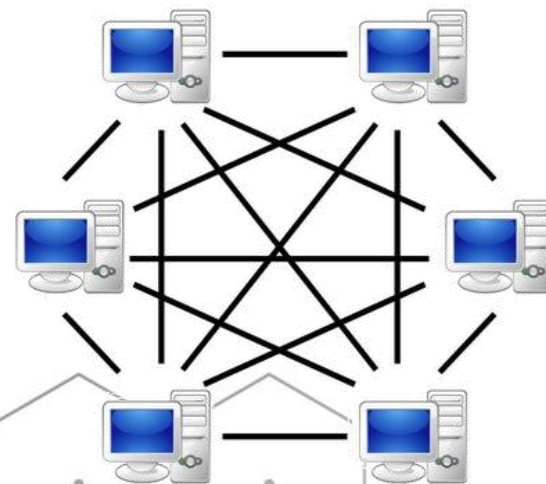
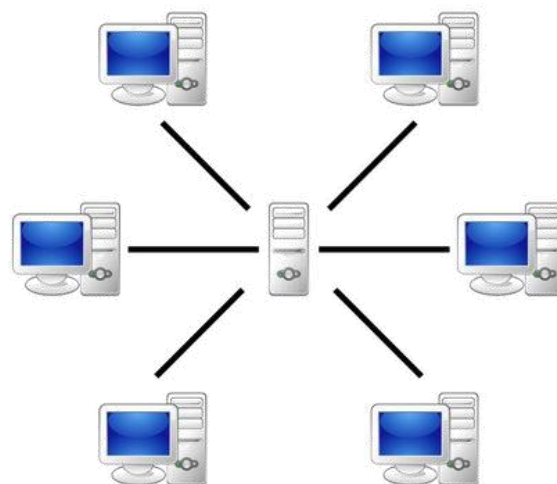
Client-Server Architecture

- **Definition:** The **Server** provides services that **Clients** request and use.
- **Control Mechanism:** The **Server** responds to client requests.
- **Communication:** Mostly **two-way**; clients request services, and the server responds.



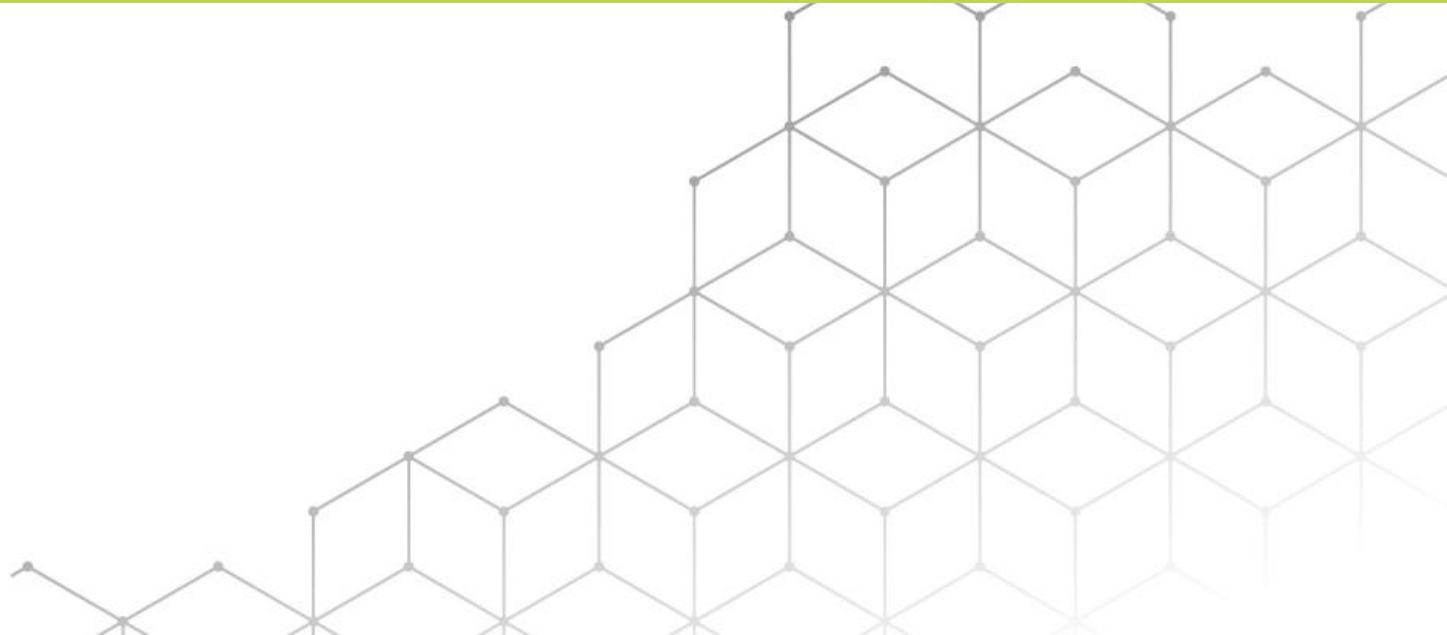
Peer to Peer Architecture

- Have no central servers
- Individual devices can be servers or clients
- Each computer has equal responsibility
- More resilient since failure of one node doesn't cripple the system.



Agenda for Today...

- Computing Resources
- Computational Scalability
- Distributed Computing
- Components of Distributed Computing
 - Cluster
 - Grids
 - Cloud Computing
- Examples



Cluster?

A **cluster** is a group of computers **physically connected** and working together as a **single system**. **Tightly coupled** (low-latency).

Example: A **supercomputer** built from multiple connected servers.



Grid?

A **grid** consists of computers that are **geographically distributed** but work together **when needed**. **Loosely coupled** (higher latency).

Example: SETI@home (search for extraterrestrial life) uses volunteers' PCs to process radio signals.



Cloud Computing

- Cloud computing is the **delivery of computing services (such as storage, processing power)** over the internet instead of using local machines or on-premise servers.
 - With cloud computing, users can **access resources on demand** without needing to own or maintain physical infrastructure.
 - Distributed computing can be implemented **without** cloud computing (e.g., a university research cluster).
- 

Market Share

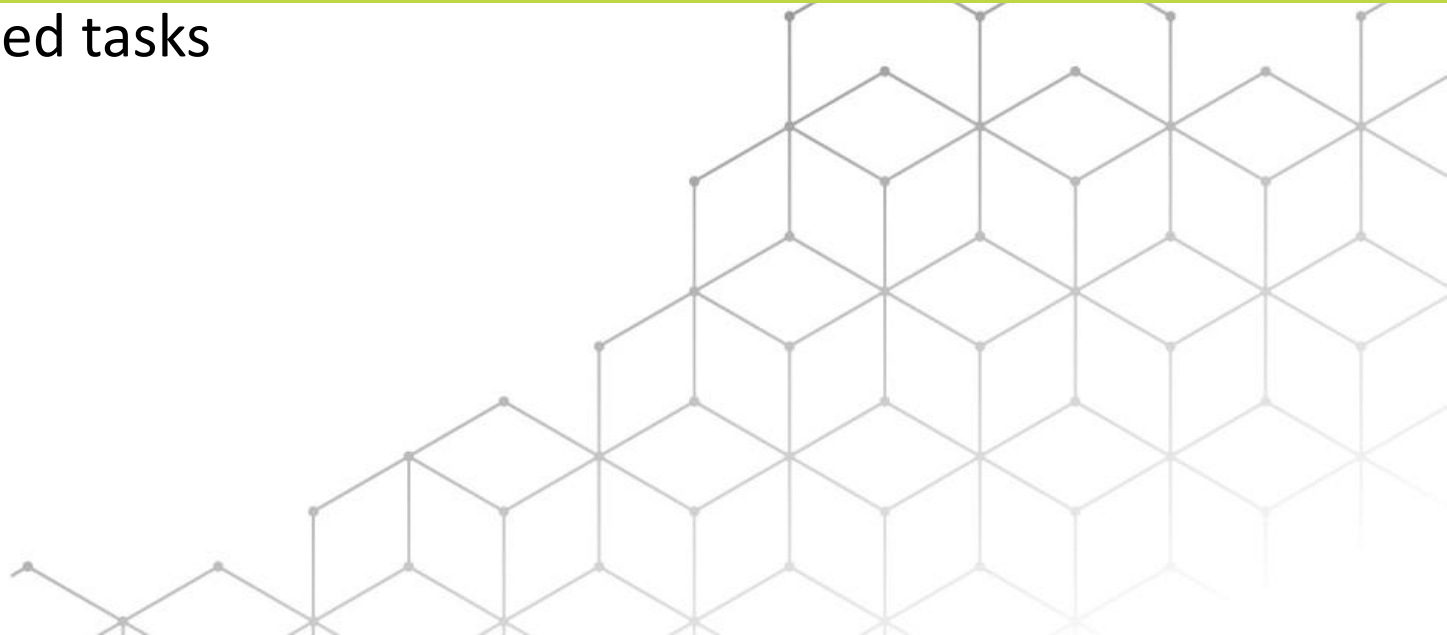
Provider	Market Share	Strengths
AWS	 32% (Leader)	Largest cloud provider, extensive services, trusted by enterprises
Azure	 22%	Strong in enterprise and hybrid cloud solutions
Google Cloud	 11%	Best for AI/ML, data analytics, and research

Example:

- Netflix, Airbnb, and NASA use AWS.
- BMW, Adobe, and LinkedIn use Azure.
- Spotify, Snapchat, and Google (YouTube, Gmail) use Google Cloud.

Agenda for Today...

- Computing Resources
- Computational Scalability
- Distributed Computing
- Examples:
 - Disk, memory and CPU bounded tasks



Task Manager: What was I doing?

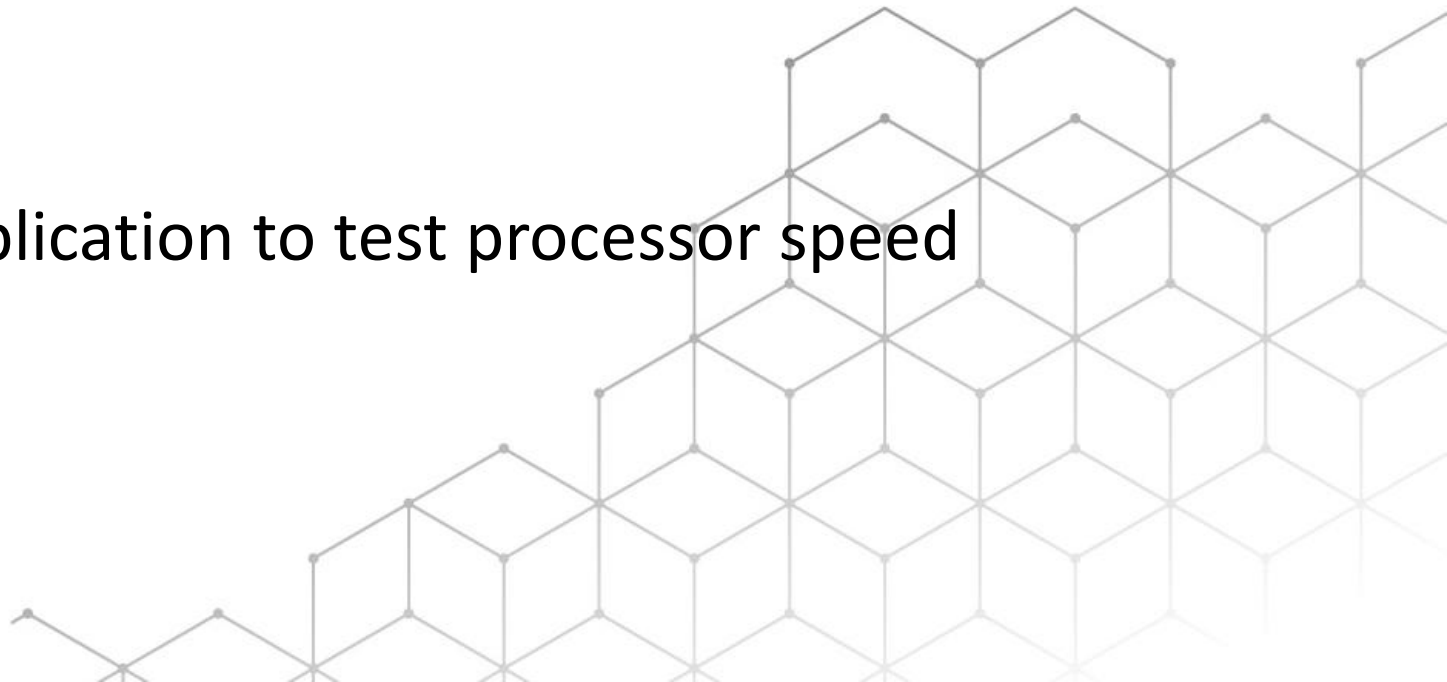
		11%	55%	1%	0%
		CPU	Memory	Disk	Network
Name	Status				
Apps (7)					
> Microsoft Edge (43)		0.1%	1,220.1 ...	0.1 MB/s	0.1 Mbps
> Microsoft Outlook (2)		0%	160.8 MB	0.1 MB/s	0 Mbps
> Microsoft PowerPoint (2)		0%	293.1 MB	0 MB/s	0 Mbps
> Microsoft Teams (10)		1.7%	986.7 MB	0.1 MB/s	2.3 Mbps
> Microsoft Word		0%	47.9 MB	0 MB/s	0 Mbps
> PyCharm Professional Editi...		0%	2,275.2 ...	0.1 MB/s	0 Mbps
> Task Manager		0.3%	89.1 MB	0.1 MB/s	0 Mbps

Task Manager: What was I doing?

		11%	55%	1%	0%
		CPU	Memory	Disk	Network
Name		Status			
Apps (7)					
>	Microsoft Edge (43)		1,220.1 ...	0.1 MB/s	0.1 Mbps
>	Microsoft Outlook (2)		160.8 MB	0.1 MB/s	0 Mbps
>	Microsoft PowerPoint (2)		293.1 MB	0 MB/s	0 Mbps
>	Microsoft Teams (10)		986.7 MB	0.1 MB/s	2.3 Mbps
>	Microsoft Word		47.9 MB	0 MB/s	0 Mbps
>	PyCharm Professional Editi...		2,275.2 ...	0.1 MB/s	0 Mbps
>	Task Manager		89.1 MB	0.1 MB/s	0 Mbps

CPU-bound Task

- A **CPU-bound task** is a type of computational task that is **limited by the processing power of the CPU** rather than other system resources like disk speed or memory bandwidth.
- These tasks require **heavy computations** rather than waiting for data input/output operations.
- Example: Large matrix multiplication to test processor speed



Example Code of CPU-bound Task

```
print("Running CPU-bound task (Matrix Multiplication)")
size = 1000 # Adjust size based on your system
A = np.random.rand(size, size)
B = np.random.rand(size, size)
start_time = time.time()
C = np.dot(A, B)
end_time = time.time()
print(f"CPU-bound task completed in {end_time - start_time:.2f} seconds\n")
```

Example of Memory-bound Task

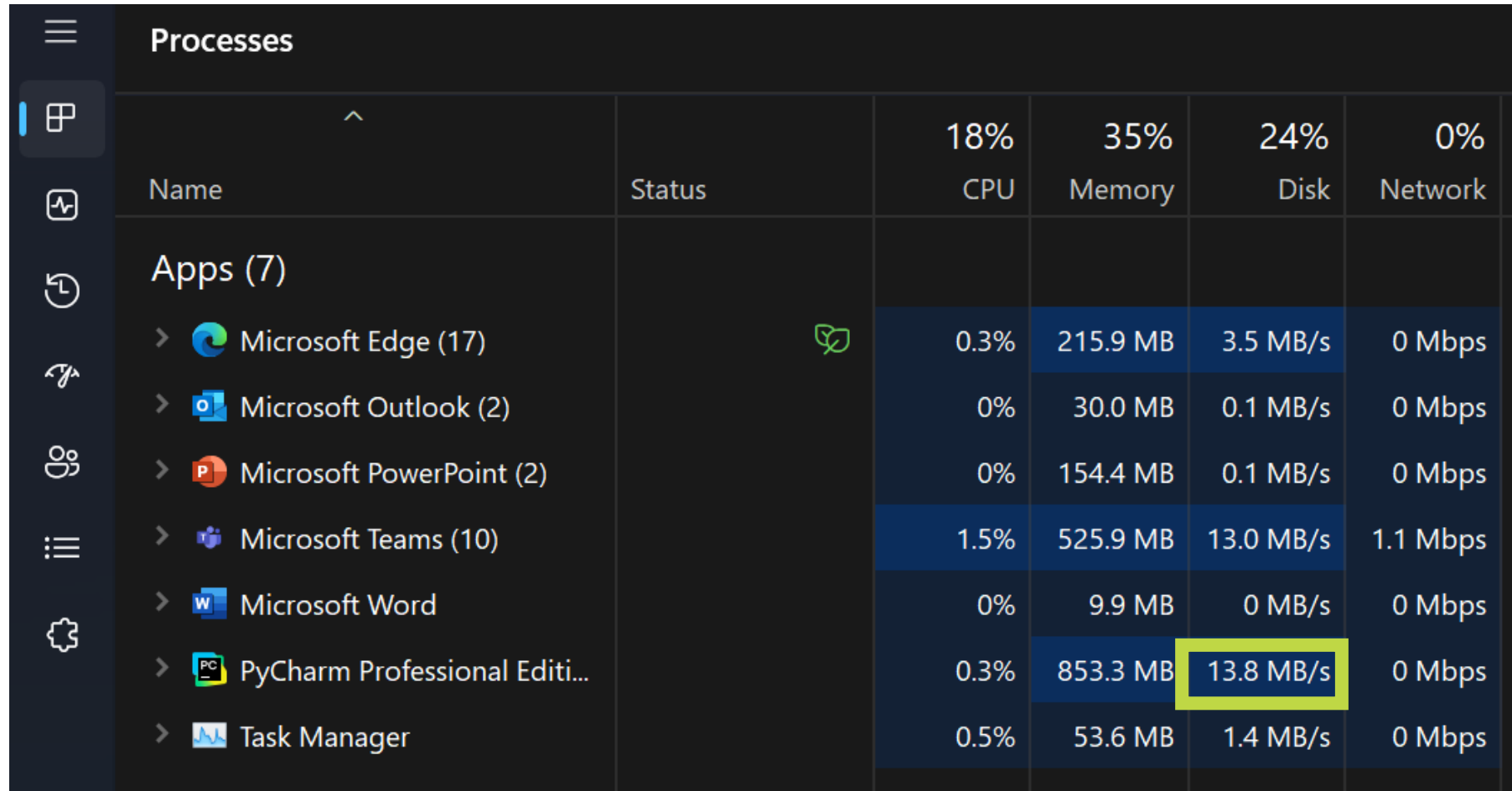
```
def memory_bound_task():  
    print("Running Memory-bound task (Large Array Manipulation)...")  
    size = 100_000_000 # 100 million elements  
    start_time = time.time()  
    arr = np.ones(size, dtype=np.float64) # Allocate large array  
    arr *= 2 # Modify array to stress memory bandwidth  
    end_time = time.time()  
    print(f"Memory-bound task completed in {end_time - start_time:.2f} seconds\n")
```

Example of Disk-bound Task

```
def disk_bound_task():  
    print("Running Disk-bound task (File I/O)...")  
    filename = "large_test_file.bin"  
    size = 500_000_000 # 500 MB file  
    data = np.random.bytes(size)  
  
    # Writing to disk  
    start_time = time.time()  
    with open(filename, "wb") as f:  
        f.write(data)  
    write_time = time.time()  
    print(f"Writing completed in {write_time -  
start_time:.2f} seconds")
```

```
# Reading from disk  
with open(filename, "rb") as f:  
    content = f.read()  
    read_time = time.time()  
    print(f"Reading completed in {read_time -  
write_time:.2f} seconds\n")  
  
# Cleanup  
os.remove(filename)
```

Disk Bounded Task on Task Manager



Processes						
			18%	35%	24%	0%
			CPU	Memory	Disk	Network
Name		Status				
Apps (7)						
>	Microsoft Edge (17)		0.3%	215.9 MB	3.5 MB/s	0 Mbps
>	Microsoft Outlook (2)		0%	30.0 MB	0.1 MB/s	0 Mbps
>	Microsoft PowerPoint (2)		0%	154.4 MB	0.1 MB/s	0 Mbps
>	Microsoft Teams (10)		1.5%	525.9 MB	13.0 MB/s	1.1 Mbps
>	Microsoft Word		0%	9.9 MB	0 MB/s	0 Mbps
>	PyCharm Professional Editi...		0.3%	853.3 MB	13.8 MB/s	0 Mbps
>	Task Manager		0.5%	53.6 MB	1.4 MB/s	0 Mbps

Next Lecture

- Big Data Platforms
 - Hadoop
 - Apache Spark



Questions?

The End

